

Core Concepts System Design - Part 1

▼ 1. Scalability

Scalability is the ability of a system to handle increased load by adding resources. The key word here is "ability", a scalable system can grow to meet demand without requiring a complete architectural overhaul.

Every successful application eventually hits the same wall: more users, more data, more requests. What worked smoothly for 1,000 users starts showing cracks at 100,000. What handled 100 requests per second crumbles at 10,000.

This is where **scalability** becomes critical.

▼ Measuring Scalability

Before scaling, you need to understand how to measure it. You cannot improve what you do not measure, and vague statements like "we need to scale" are useless without concrete numbers.

Scalability is typically evaluated along these dimensions:

Load Metrics

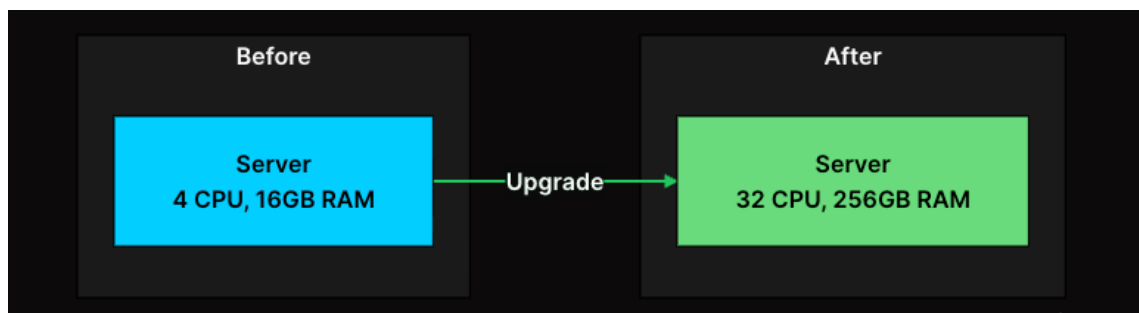
Metric	Description	Example
Requests per second (RPS)	Number of API calls the system handles	10,000 RPS
Concurrent users	Users active at the same time	50,000 concurrent
Data volume	Amount of data stored or processed	10 TB storage
Throughput	Data transferred per unit time	1 GB/s
Query rate	Database queries per second	50,000 QPS
Message rate	Messages processed through queues	100,000 msg/s

The goal is to keep performance relatively stable as load increases. Ideally, you want linear or sublinear degradation, where doubling load does not double response time. When response times spike or the system starts timing out, you have hit a scalability wall.

▼ Vertical Scaling (Scale Up)

Vertical scaling means adding more power to your existing machines. Instead of adding more servers, you upgrade to bigger ones.

This is often the first response to performance problems because it requires no architectural changes.



Common Vertical Scaling Actions

- **Add more CPU cores** for compute-intensive workloads
- **Increase RAM** to cache more data in memory
- **Use faster SSDs** to reduce I/O bottlenecks
- **Upgrade network cards** for higher bandwidth

Pros

- **Simple:** No code changes required. Just move to a bigger machine.
- **Lower latency:** All data is local, no network hops.
- **No distributed complexity:** A single server means no network partitions, no data synchronization issues.

Cons:

- **Hardware limits:** You cannot scale beyond the largest available machine. Even cloud providers have limits.
- **Single point of failure:** One server means one failure point. If it goes down, everything goes down.
- **Cost curve:** Larger machines cost disproportionately more. Doubling capacity often more than doubles cost.
- **Downtime during upgrades:** Migrating to a bigger machine typically requires downtime.

When to Use Vertical Scaling

Vertical scaling works well for:

- **Databases** where data locality matters (before sharding becomes necessary)
- **Applications with strong consistency requirements**
- **Early-stage startups** that need simplicity over scale
- **Workloads with predictable, moderate growth**

NOTE: Never dismiss vertical scaling as "not scalable." Many real-world systems run on vertically scaled databases for years. The key is knowing when horizontal scaling becomes necessary.

▼ Horizontal Scaling (Scale Out)

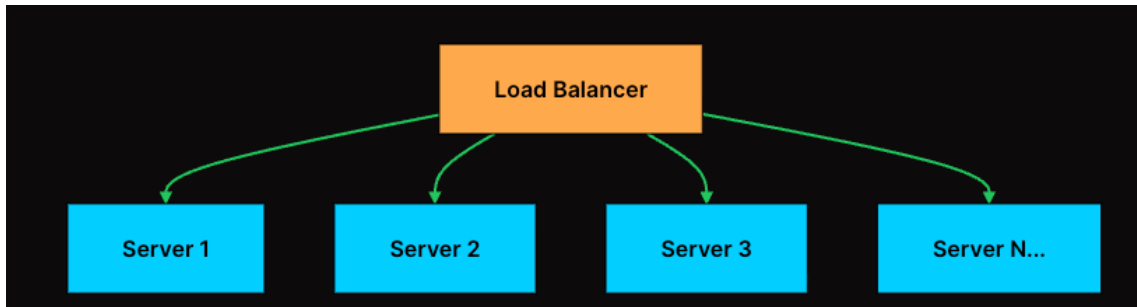
Vertical scaling eventually hits a ceiling. When the biggest available machine is not big enough, or when you need fault tolerance that a single machine cannot provide, you need a different approach.

Horizontal scaling means adding more machines rather than upgrading existing ones. Instead of one powerful server, you distribute the load across many commodity servers.

This is how companies like Google, Netflix, and Amazon handle billions of requests.

Instead of one powerful server, you have many commodity servers working together. A load balancer distributes incoming requests across all

servers.



Pros

- **No hard limit:** You can keep adding servers as needed. Cloud providers make this nearly unlimited.
- **Fault tolerance:** If one server fails, others continue serving traffic. No single point of failure.
- **Cost-effective:** Many smaller machines often cost less than one giant machine.
- **Geographic distribution:** You can place servers closer to users for lower latency.

Cons

- **Complexity:** Distributed systems are harder to build, debug, and maintain.
- **Data consistency:** Keeping data synchronized across servers is challenging.
- **Network overhead:** Communication between servers adds latency.
- **Stateless requirement:** Application servers typically need to be stateless, which may require architectural changes.

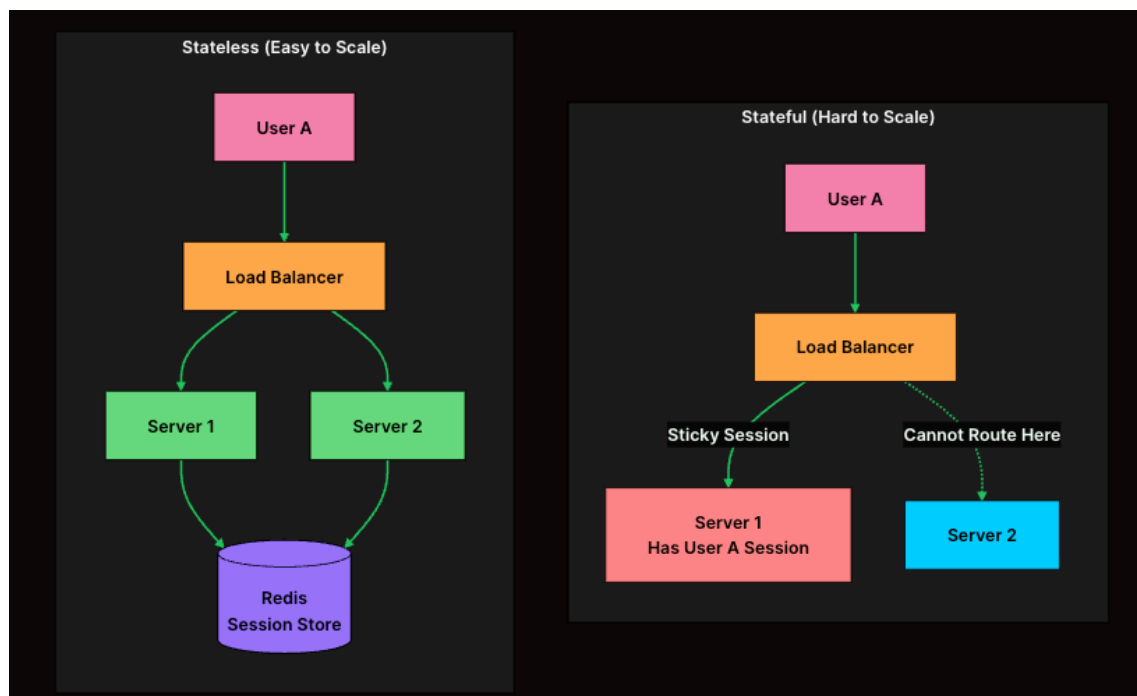
Stateless vs Stateful Services

For horizontal scaling to work effectively, services should be **stateless**. A stateless service does not store any session data locally. Each request can be handled by any server.

In the stateful model, once a user's session is stored on Server 1, all their requests must go to that same server. This creates hotspots and makes it risky to remove servers. In the stateless model, session data lives in a shared store like Redis, so any server can handle any request.

To make services stateless:

- Store session data in a shared cache (Redis, Memcached)
- Use tokens (JWT) instead of server-side sessions
- Store uploaded files in object storage (S3) instead of local disk

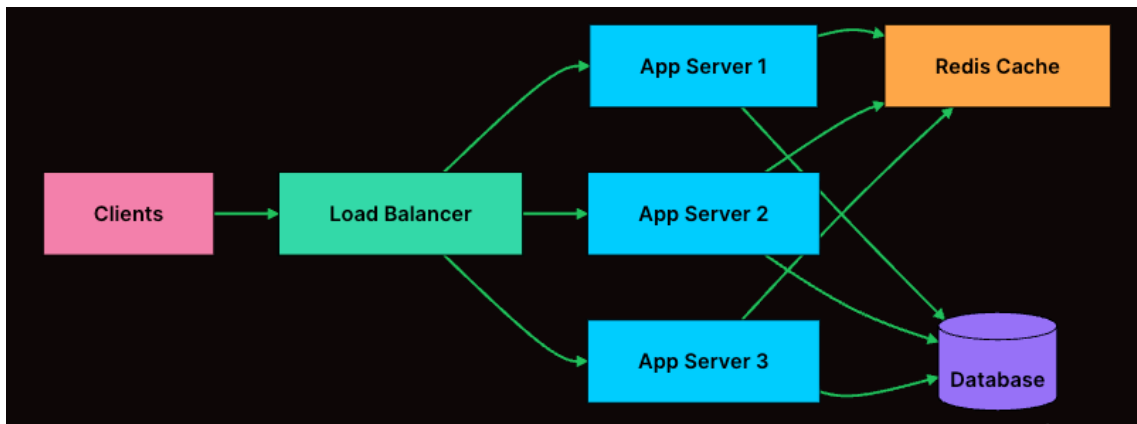


Scaling Different Components

A typical system is not monolithic. It has multiple components, each with different scaling characteristics and challenges. Understanding these differences is crucial because the scaling strategy that works for one tier often does not work for another.

▼ Application Tier

Application servers are usually the easiest to scale horizontally, provided they are stateless:



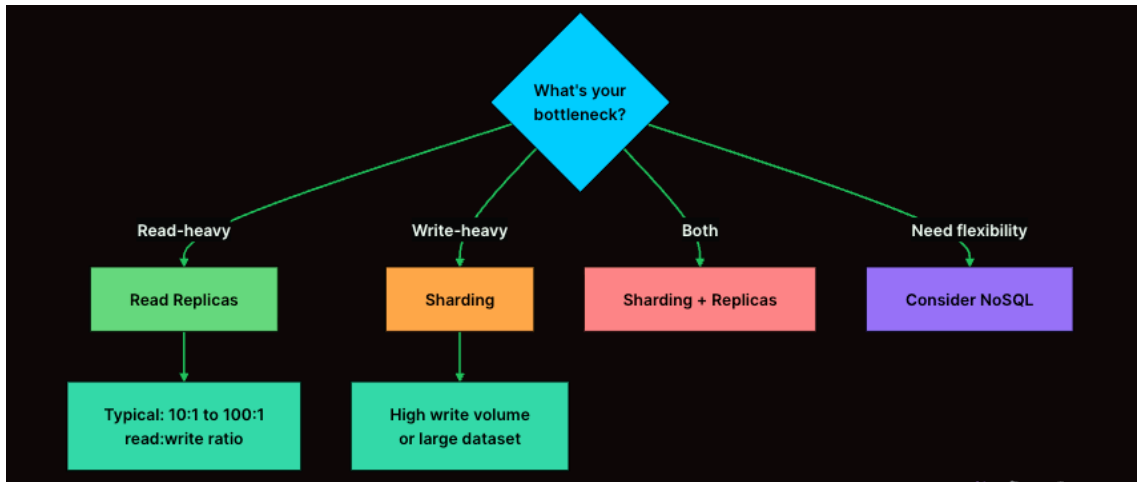
Key strategies:

- Make services stateless
- Use a load balancer to distribute traffic
- Auto-scale based on CPU, memory, or request count
- Deploy across multiple availability zones

▼ Database Tier

Databases are typically the hardest to scale because they manage state. Unlike application servers, you cannot simply spin up more database instances and put a load balancer in front of them. Data consistency, durability, and transaction isolation all complicate matters.

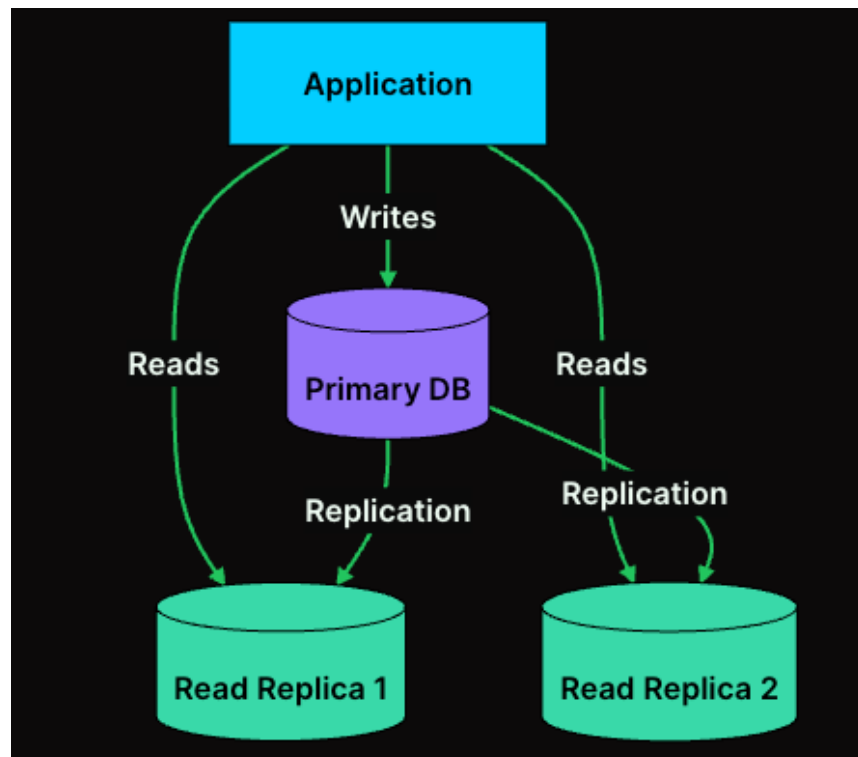
The approach depends on your workload pattern:



▼ 1. Read Replicas

For read-heavy workloads (which most applications are), create copies of your database that handle read queries:

Primary handles all writes, replicas receive changes and serve reads.



When to use: Read-to-write ratio is 10:1 or higher, and writes are not the bottleneck.

Pros

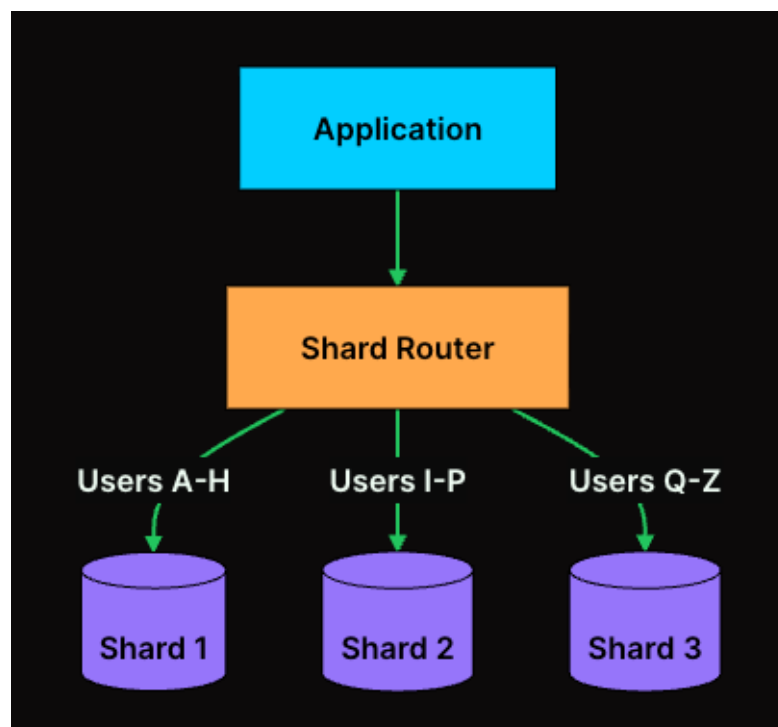
- Simple to set up (managed services handle it)
- Offloads read traffic from primary
- Provides read availability if primary fails
- No application changes for basic setup

Cons

- Does not help with write-heavy workloads
- Introduces replication lag (stale reads)
- Replicas consume storage (full data copy)
- Failover can cause brief inconsistency

▼ 2. Sharding (Partitioning)

When read replicas are not enough, or when write volume exceeds what a single primary can handle, you need to split your data across multiple databases based on a partition key:



Pros

- Distributes both reads AND writes
- Scales horizontally (add more shards)
- Each shard is smaller, faster
- Can place shards in different regions

Cons

- Complex to implement correctly
- Cross-shard queries are expensive or impossible
- Rebalancing shards is operationally difficult
- Transactions across shards are very hard

Common sharding strategies:

- **Range-based:** Shard by value ranges (A-H, I-P, Q-Z)
- **Hash-based:** Hash the key and mod by number of shards
- **Directory-based:** Maintain a lookup table mapping keys to shards

▼ 3. NoSQL Databases

NoSQL databases like Cassandra, MongoDB, and DynamoDB are designed for horizontal scaling from the ground up:

- **Built-in sharding:** Data is automatically distributed
- **Eventual consistency:** Trade strong consistency for availability
- **No joins:** Data model must accommodate denormalization

Pros

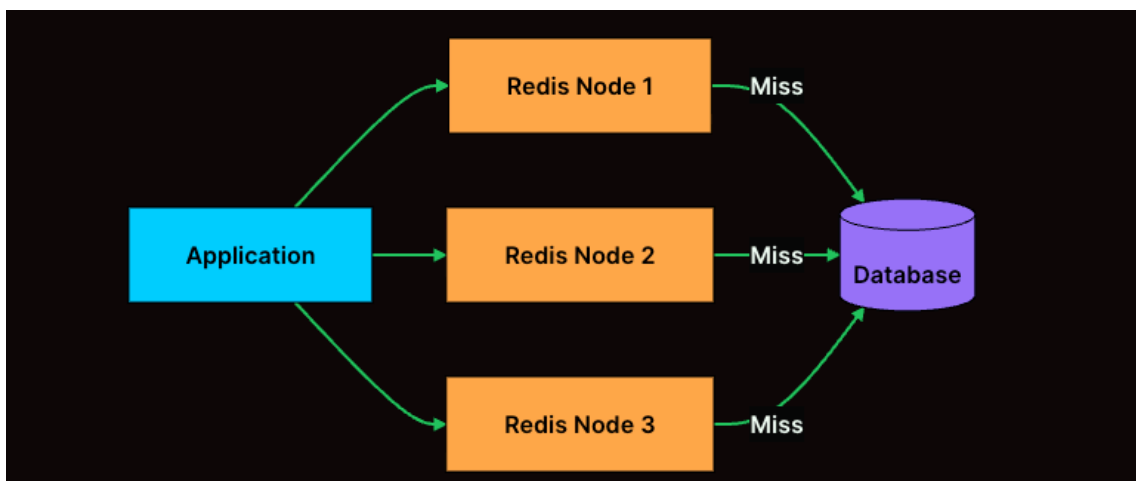
- Built-in sharding (automatic distribution)
- Designed for horizontal scale
- Often better write performance
- Schema flexibility

Cons

- Different query patterns than SQL
- No joins (denormalization required)
- Eventual consistency in many cases
- Less tooling ecosystem than SQL

▼ Caching Tier

Caching reduces load on databases and improves response times. A well-designed cache can handle 100x the throughput of a database, making it essential for high-traffic systems. Redis, for example, can handle 100,000+ operations per second on a single node.



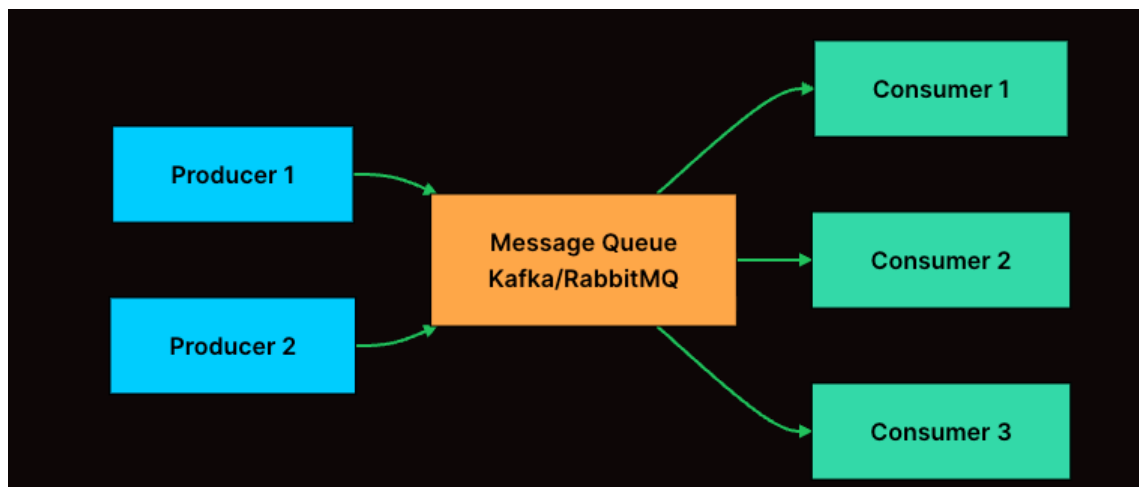
Cache Scaling strategies:

- **Redis Cluster:** Automatically partitions data across nodes using hash slots
- **Consistent hashing:** Distributes keys evenly and minimizes redistribution when nodes are added or removed
- **Cache-aside pattern:** Application checks cache first, falls back to database on cache miss, then populates the cache

▼ Message Queue Tier

Message queues are essential for scaling asynchronous workloads. They decouple producers from consumers, allowing each to scale

independently, and they buffer traffic spikes so consumers can process at their own pace.



How queues help scalability:

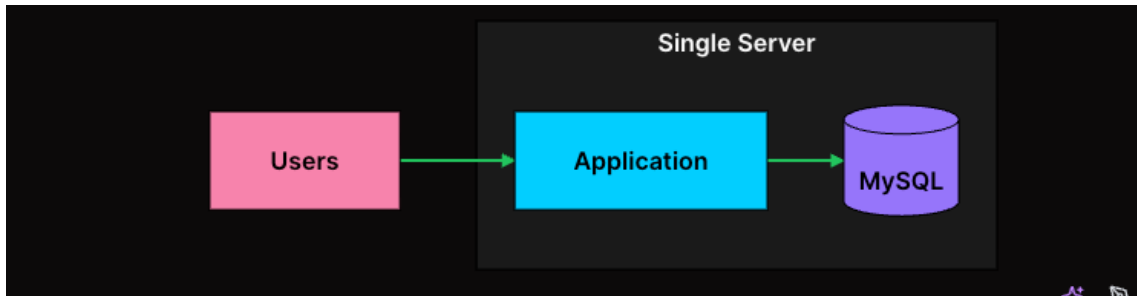
- **Decouple producers and consumers:** Scale each independently
- **Buffer traffic spikes:** Queue absorbs bursts, consumers process at their own pace
- **Partition topics:** Kafka partitions allow parallel consumption

Example: Scaling from 0 to millions of users

Let us walk through how a startup might scale a social media application from zero to millions of users. Each stage solves a specific bottleneck that emerged from growth.

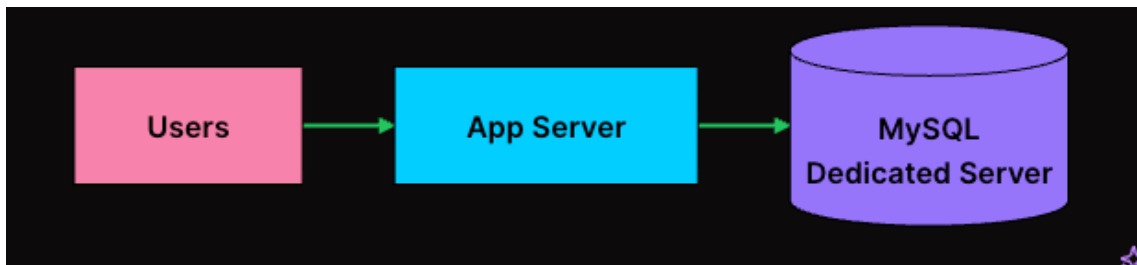
▼ Stage 1: Single Server (0-10K users)

At launch, everything runs on one machine. The application and database share the same server. This setup is simple, cheap, and perfectly adequate for a few thousand users. There is no distributed system complexity, no network latency between components, and debugging is straightforward.



The bottleneck emerges when the application and database start competing for CPU and memory on the same machine.

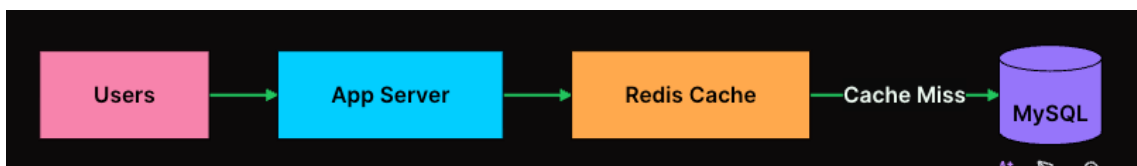
▼ Stage 2: Separate Database (10K-100K users)



The first scaling move is usually separating the database onto its own machine. Now each component can be tuned independently. You can give the database server more RAM for caching, while the app server gets more CPU for request processing.

The bottleneck shifts to the database. As user counts grow, the database handles more queries, and read operations start slowing down.

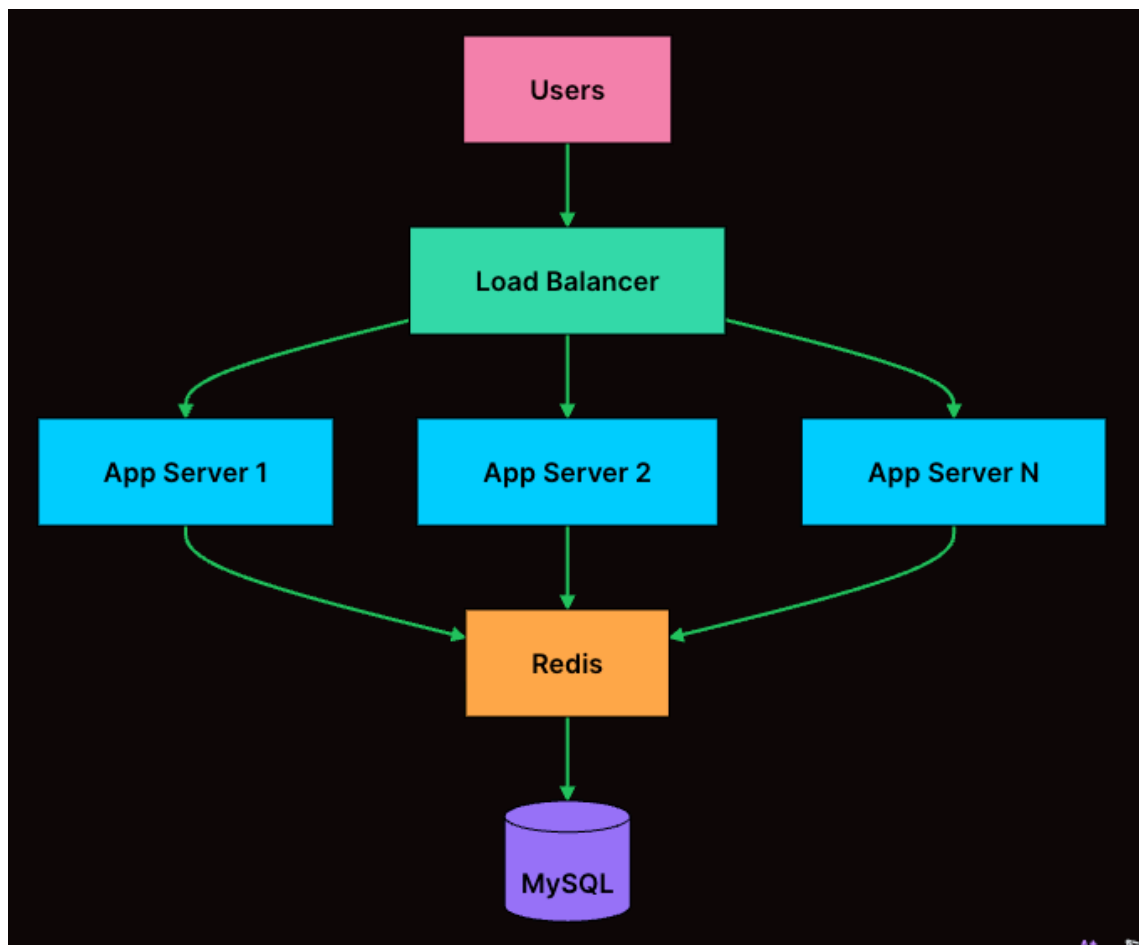
▼ Stage 3: Add Caching (100K-500K users)



Adding a cache layer dramatically reduces database load. Hot data, things like user profiles, recent posts, and session data, gets served from memory. Redis can handle hundreds of thousands of reads per second, far more than MySQL. With a good caching strategy, 80-90% of reads never hit the database.

The bottleneck is now the single app server. It cannot handle the incoming request volume.

▼ **Stage 4: Multiple App Servers (500K-2M users)**

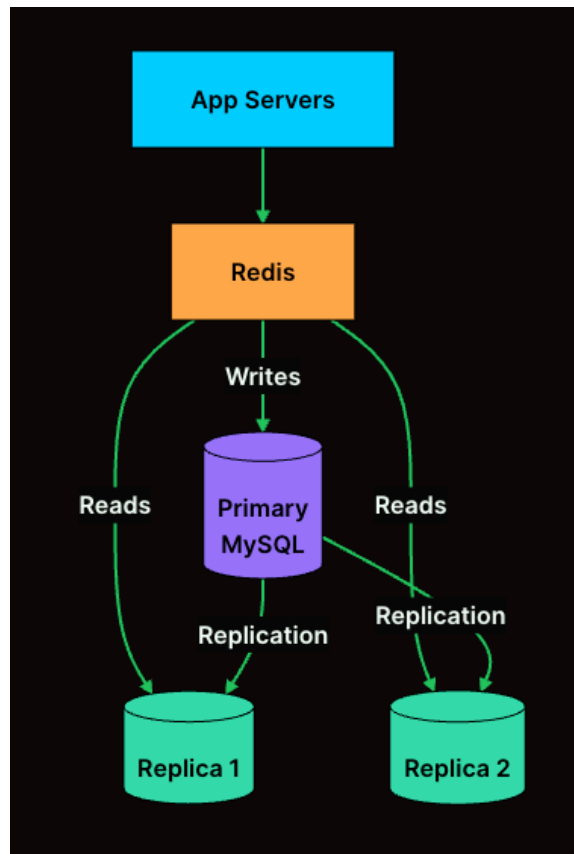


This is where horizontal scaling begins. A load balancer distributes traffic across multiple app servers. Each server is stateless, storing no session data locally. The Redis cache serves as the shared session store.

Adding more app servers is now trivial. Need more capacity? Spin up another server. Traffic spike during peak hours? Auto-scaling adds servers automatically.

The bottleneck shifts back to the database. With more app servers generating more queries, the single MySQL instance becomes overwhelmed.

▼ Stage 5: Read Replicas (2M-10M users)

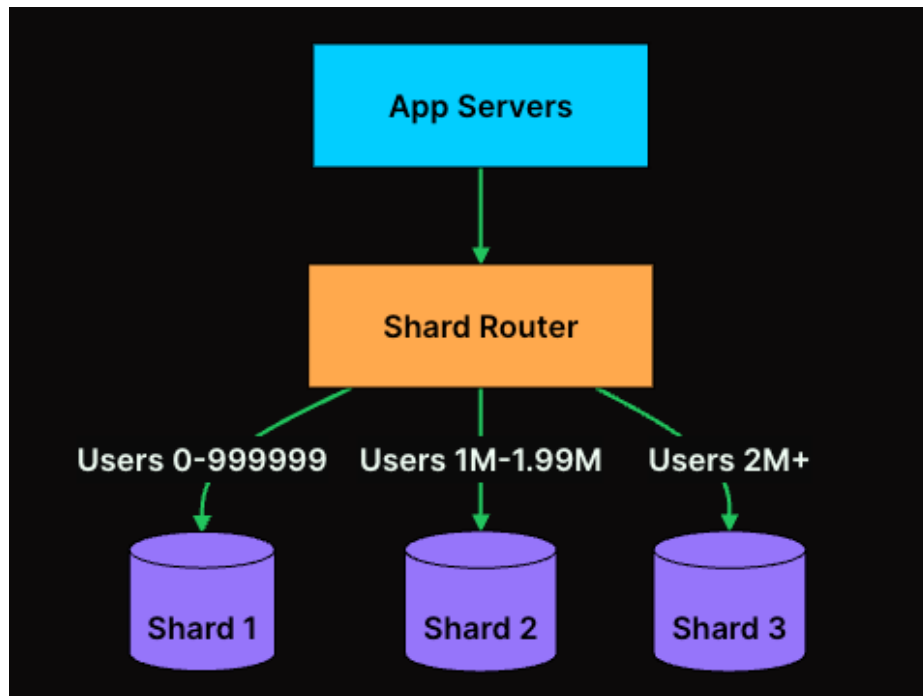


Most applications are read-heavy, with reads outnumbering writes by 10:1 or more. Read replicas take advantage of this pattern. The primary database handles all writes, while replicas serve read queries. This multiplies read capacity without changing the application much.

The trade-off is replication lag. Replicas may be a few milliseconds behind the primary, so recently written data might not be immediately visible on reads. For most applications, this is acceptable.

The bottleneck becomes write throughput. One primary database can only handle so many writes per second.

▼ Stage 6: Sharding (10M+ users)



Sharding is the final frontier of relational database scaling. Data is partitioned across multiple databases based on a shard key, typically user ID. Each shard handles a subset of users, distributing both read and write load.

This is powerful but comes with significant complexity. Cross-shard queries become expensive or impossible. Rebalancing shards when they grow unevenly is operationally challenging. Many teams at this stage consider moving to distributed databases like CockroachDB or Vitess that handle sharding automatically.

Summary

Scalability is about designing systems that can grow with demand without falling apart. Here are the key takeaways:

- **Vertical scaling** is simple but has limits. Use it for databases, early-stage systems, and workloads where simplicity matters more than infinite growth.
- **Horizontal scaling** is more complex but can grow indefinitely. It requires stateless services and careful data management.

- **Different components scale differently.** App servers are easy to scale horizontally. Databases are hard because they manage state. Know the difference.
- **Always identify the bottleneck first** before deciding how to scale. Adding more app servers does not help if the database is the problem.
- **Common patterns** like load balancing, caching, async processing, and database optimization appear in almost every scalable system.

Scalability addresses the question of handling more load. But a system that can handle a million users is not very useful if it goes down every week. A scaled system that crashes still fails your users.

This brings us to our next topic: **availability**. How do you ensure your system stays operational even when individual components fail?

▼ 2. Availability

Scaling solves only half the problem. What good is a system that can handle millions of requests if it crashes when a single server fails? This is where **availability** comes in.

What Availability Measures: Availability measures how often your system is operational and accessible to users. A highly available system continues functioning even when individual components fail.

One important distinction before we begin: **availability** is not the same as **reliability**. A system can be highly available (always up) but unreliable (sometimes gives wrong answers).

▼ Measuring Availability

Availability is typically expressed as a percentage of uptime over a given period. The formula is straightforward:

$$\text{Availability} = \text{Uptime} / (\text{Uptime} + \text{Downtime})$$

For example, if a system was up for 364 days and down for 1 day in a year:

$$\text{Availability} = 364 / 365 = 99.73\%$$

That single day of downtime drops you below "three nines" availability. When you frame it as "one day," it sounds acceptable.

The "Nines" of Availability

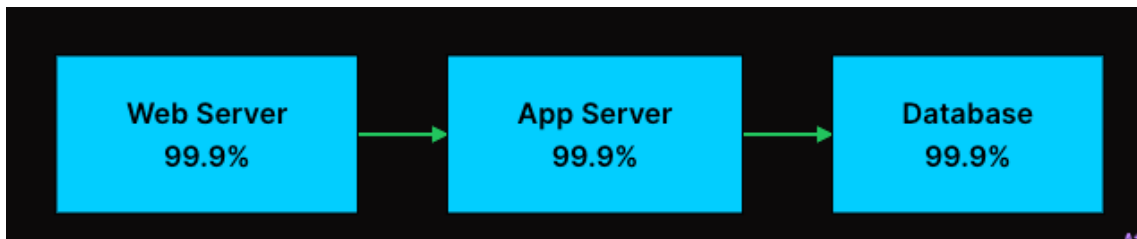
Availability is often described in terms of "nines." Each additional nine dramatically reduces allowed downtime:

Availability in Series vs Parallel

How you combine components dramatically affects overall availability.

Components in Series

When components are in series, meaning all must work for the system to function, availability reduces:

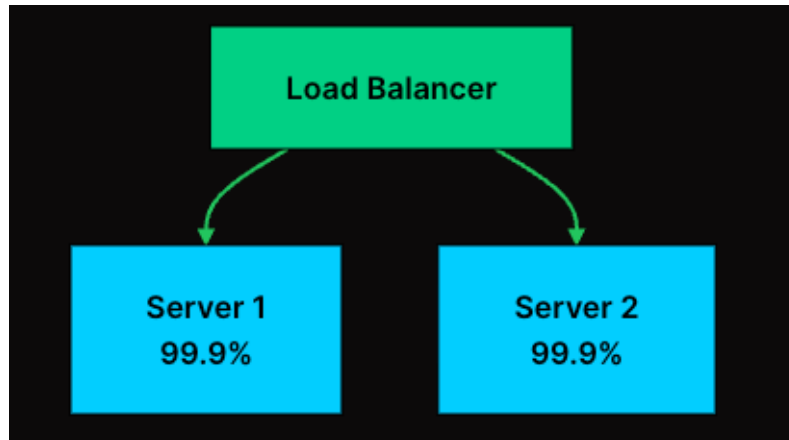


Overall = 99.9% × 99.9% × 99.9% = 99.7%

Each component in the chain reduces overall availability. You started with three components, each at "three nines," but the combined system is below three nines. Add more components in series, and availability keeps dropping.

Components in Parallel

When components are in parallel, meaning any can handle the request, availability improves dramatically:



For both servers to be down simultaneously, both must fail at the same time:

Failure probability = $0.1\% \times 0.1\% = 0.0001\%$

Availability = $100\% - 0.0001\% = 99.9999\%$

Two servers with 99.9% availability each give you nearly six nines when running in parallel. This is the power of redundancy.

▼ Common Failure Modes

To design for availability, you must understand how things fail. Failures do not ask permission, and they rarely happen at convenient times. Knowing the common failure modes helps you prepare for them.

Hardware Failures

Everything physical eventually breaks. The question is when, not if.

At scale, hardware failures are not exceptional events. They are routine. A data center with 10,000 servers will see hundreds of hardware failures per year. If your architecture cannot handle a server dying at any moment, you do not have a highly available system.

Software Failures

Hardware breaks randomly. Software breaks creatively.

- **Bugs:** Code defects that cause crashes or incorrect behavior

- **Memory leaks:** Gradual resource exhaustion
- **Deadlocks:** Processes waiting on each other indefinitely
- **Cascading failures:** One failure triggering failures in dependent systems

Network Failures

Networks fail in ways that are subtle, intermittent, and painful to debug.

- **Packet loss:** Data does not reach its destination
- **Latency spikes:** Delays in communication
- **Partition:** Network split isolates groups of servers
- **DNS failures:** Name resolution stops working

Human Errors

Here's the uncomfortable reality: many studies attribute **70–80% of outages** to human error, not hardware, not software, but people.

Common examples:

- **Configuration mistakes:** wrong environment variable, typo in a config file
- **Failed deployments:** bad code or broken migrations pushed to production
- **Accidental deletions:** running the wrong command in the wrong place
- **Capacity planning errors:** underestimating traffic for a launch

This is why automation, testing, and guardrails matter so much. Humans make mistakes. Good systems make those mistakes hard to make and easy to recover from.

Redundancy: The Foundation of Availability

If there is one concept that underpins all of availability, it is redundancy. The logic is simple: if you have only one of something, when it fails, you have zero. If you have two, when one fails, you still have one.

Redundancy means having backup components that can take over when primary components fail.

▼ Active-Passive (Standby)

In an active-passive configuration, one component handles all the work while another waits idle as a backup. When the active component fails, the passive one takes over.



Active-passive mode is commonly used in situations where you want a single source of truth and controlled writes like databases, stateful services, and systems requiring a single leader.

Pros

- Simple to reason about
- Standby typically uses fewer resources
- Clear source of truth

Cons

- Failover takes time (detection + promotion + routing changes)
- Standby may not be truly “production-ready” because it isn’t tested under real load
- Potential for split-brain problem

The standby can be configured in different states of readiness:

Standby Type	State	Failover Time	Cost
Cold Standby	Powered off, needs to boot	Minutes	Lowest
Warm Standby	Running but not receiving traffic	Seconds to minutes	Medium
Hot Standby	Running, data synchronized, ready to serve	Seconds	Highest

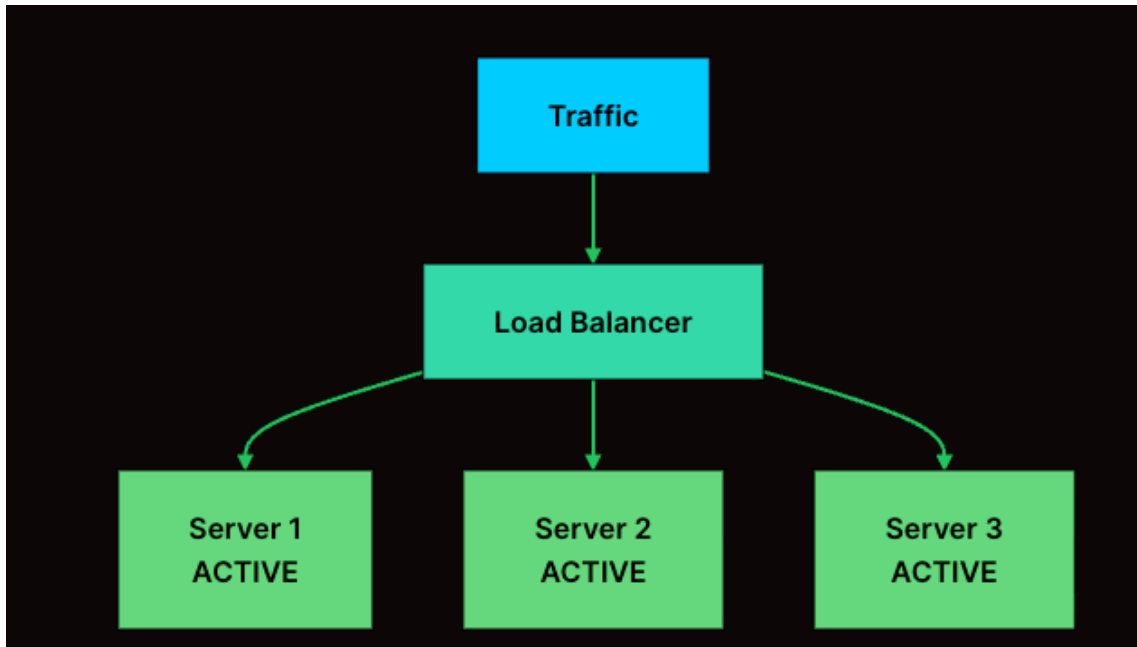
Cold standby is cheapest but slowest. The backup server is not running, so failover requires booting the machine, starting services, and potentially restoring data. This might take 5-15 minutes, which is too slow for most production systems but acceptable for disaster recovery.

Warm standby keeps the backup running and configured, but not actively processing requests. It might be receiving replicated data but is not in the load balancer pool. Failover involves adding it to the pool and possibly promoting it, which takes seconds to a few minutes.

Hot standby is the most expensive but fastest. The backup is fully synchronized and ready to serve immediately. For databases, this often means synchronous replication where every write is confirmed on both primary and standby before acknowledging the client.

▼ Active-Active

In an active-active configuration, all components handle traffic simultaneously. There is no distinction between primary and backup because every node is doing real work.



When one node fails, the load balancer simply stops sending traffic to it. There is no failover process because the other nodes were already handling traffic. The remaining nodes absorb the additional load.

Pros

- No failover delay
- All nodes tested under real load
- Better resource utilization

Cons

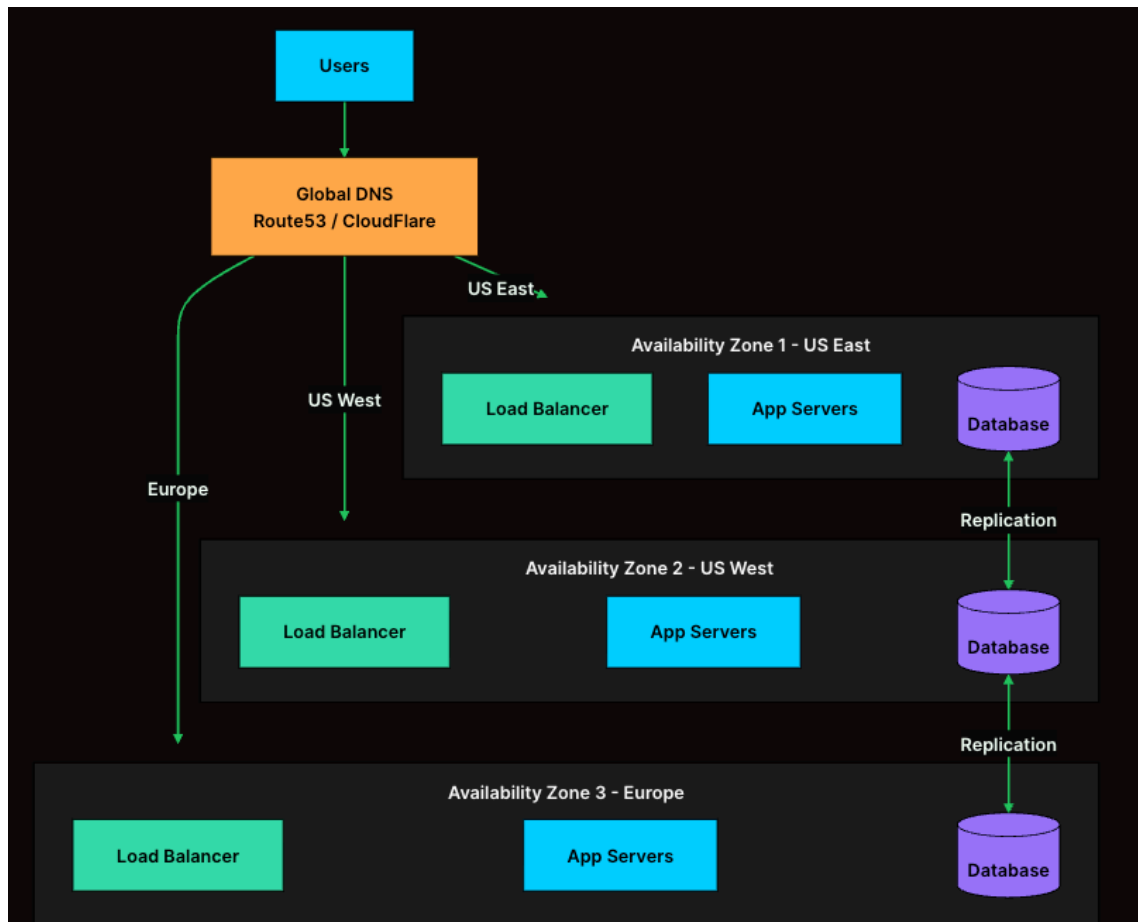
- More complex
- Must handle data consistency across nodes
- requires stateless design or shared state

The key requirement for active-active is that requests can be handled by any node. This works naturally for stateless services where each request is independent. For stateful services, you need either shared storage (like a database or Redis) or sticky sessions (which reduces availability benefits).

▼ Geographic Redundancy

Redundancy within a single data center protects against hardware failures, but what if the entire data center goes offline? Power outages, network cuts, natural disasters, or even a backhoe cutting a fiber line can take down an entire facility.

Geographic redundancy distributes your system across multiple physical locations:



Cloud providers offer different levels of geographic redundancy:

Level	What It Is	Protects Against	Latency Impact
Availability Zones (AZs)	Separate data centers in the same region, connected by low-latency links	Single data center failure	Minimal (1-2ms)

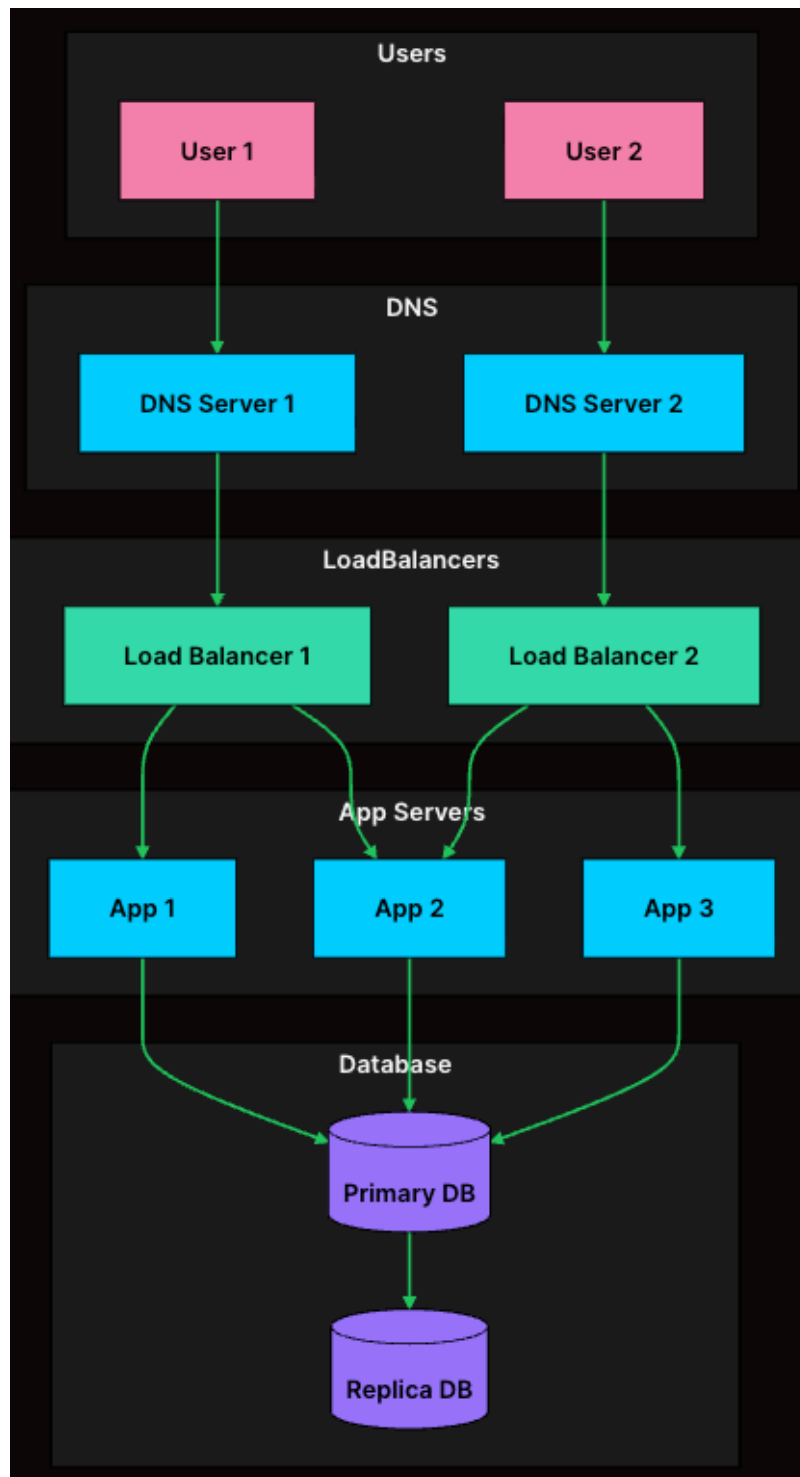
Level	What It Is	Protects Against	Latency Impact
Regions	Geographically separate areas (e.g., US-East vs US-West)	Regional disasters, widespread outages	Significant (50-100ms+)
Multi-Cloud	Different cloud providers (AWS + GCP)	Cloud provider outages	Variable

Availability Zones are the sweet spot for most applications. They provide meaningful isolation (separate power, cooling, and network) while keeping latency low enough for synchronous replication. Most cloud-native applications deploy across at least two AZs.

Multi-region deployment is necessary for global applications or those requiring disaster recovery from regional events. The challenge is data replication, since synchronous replication across regions adds significant latency. Most multi-region systems use asynchronous replication and accept some data loss in a disaster (typically seconds to minutes of transactions)

▼ Redundancy Across Layers

A chain is only as strong as its weakest link. If you have redundant app servers but a single database, the database is your single point of failure. True high availability requires redundancy at every layer of your stack.



Notice that redundancy gets harder as you move down the stack. Adding more web servers is trivial. Adding database replicas with automatic failover requires careful engineering.

Note

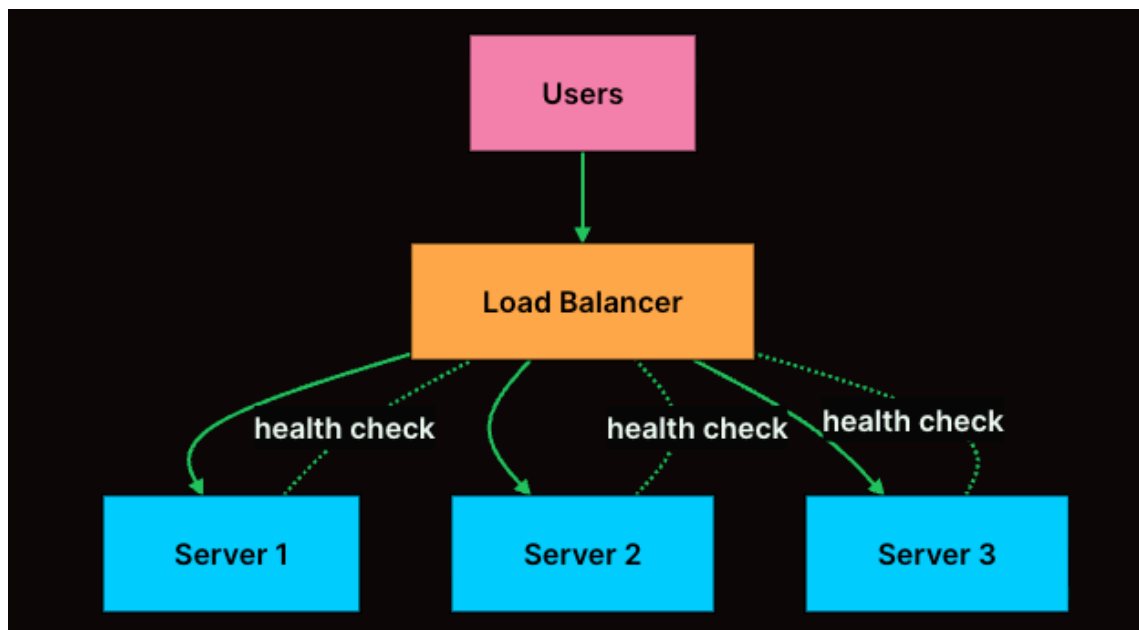
Redundancy is not free. Every backup server, every replica, every additional availability zone costs money. The question is whether that cost is justified by the reduction in downtime risk.

High Availability Patterns

Patterns are reusable solutions to common problems. The following patterns appear repeatedly in highly available systems.

▼ Pattern 1: Load Balancer with Multiple Backends

The most common and fundamental pattern for stateless services. A load balancer distributes traffic across multiple servers, automatically routing around failures.

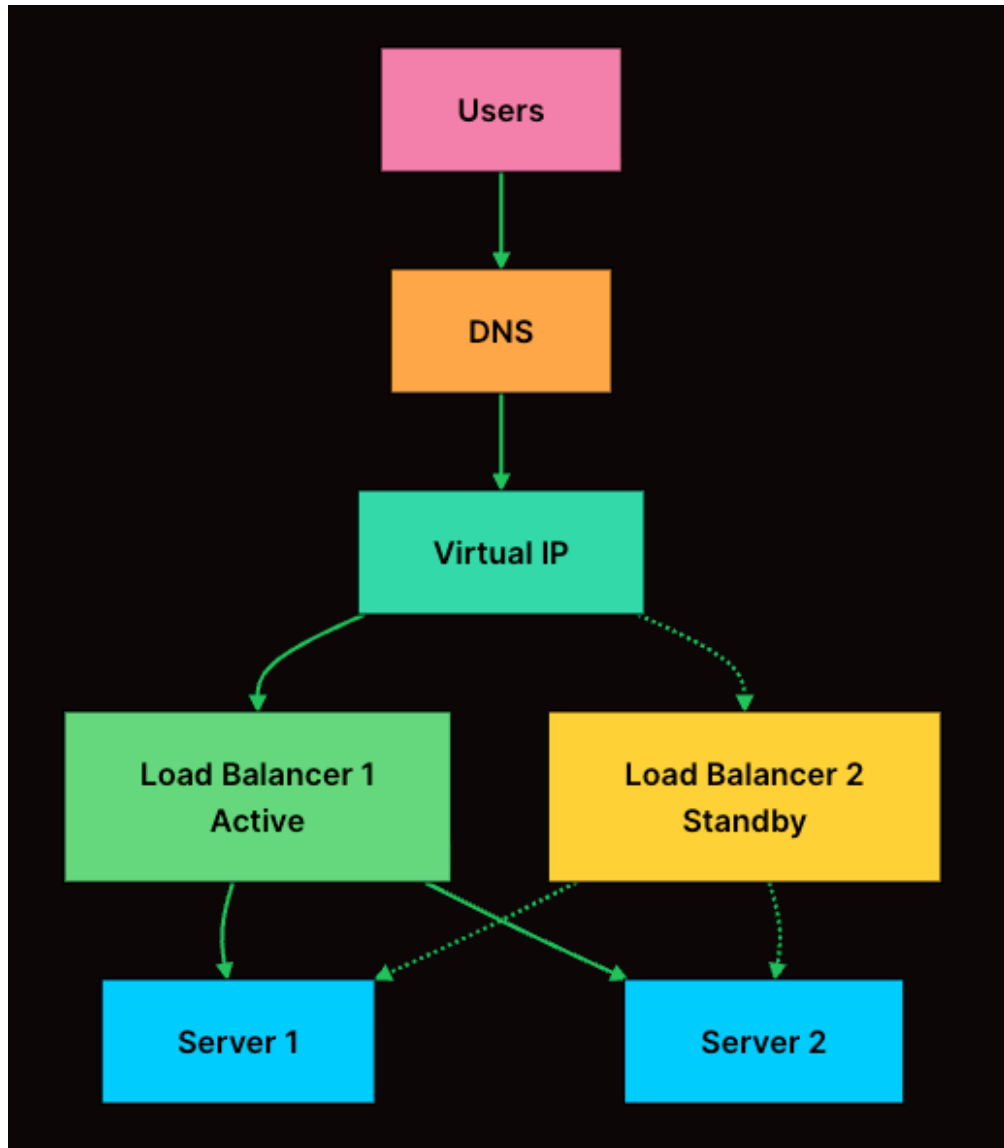


How it provides high availability:

- Load balancer continuously monitors backend health
- Failed servers are automatically removed from rotation
- Traffic redistributes to healthy servers within seconds
- New servers can be added without any downtime

Load Balancer Redundancy

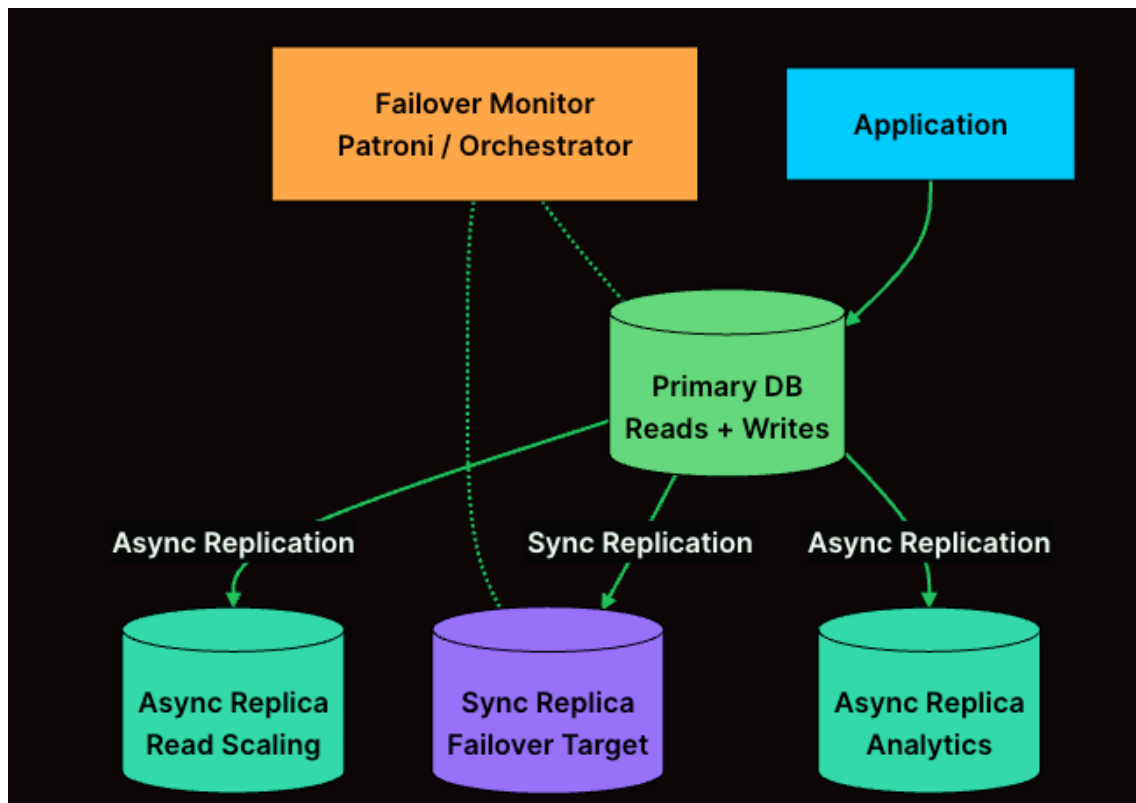
The load balancer itself is a single point of failure. For true high availability, you need redundant load balancers:



Cloud providers handle this automatically. AWS ALB, Google Cloud Load Balancer, and Azure Load Balancer are all managed services with built-in redundancy. On-premises, you might use keepalived with a virtual IP that floats between two HAProxy instances.

▼ Pattern 2: Database Replication with Automatic Failover

Databases are stateful and cannot simply be load-balanced like web servers. Database high availability requires replication and careful failover management.



Synchronous vs Asynchronous Replication

Type	How It Works	Data Loss	Performance Impact
Synchronous	Write confirmed only after replica acknowledges	Zero (RPO = 0)	Higher latency (wait for replica)
Asynchronous	Write confirmed immediately, replica catches up later	Possible (seconds to minutes)	No impact on write latency
Semi-synchronous	Wait for at least one replica, others async	Minimal	Moderate impact

Synchronous replication guarantees zero data loss but adds latency. Every write must wait for the replica to confirm. If your replica is in a

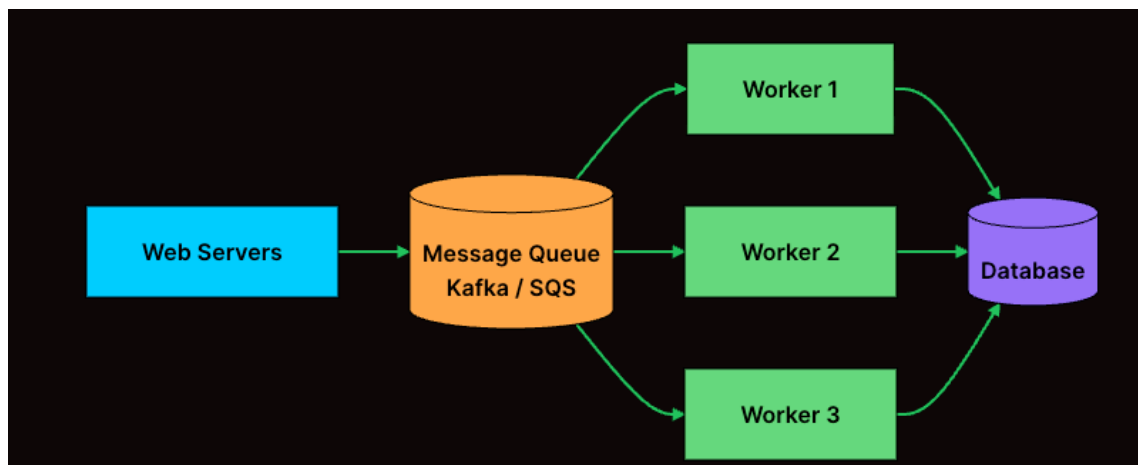
different region, this adds significant latency (50-100ms per write).

Asynchronous replication has no performance impact but can lose data. If the primary fails, any writes not yet replicated are lost. The "replication lag" is typically seconds but can grow during high load.

Most production systems use synchronous replication for the failover target and asynchronous replication for read replicas and analytics.

▼ Pattern 3: Queue-Based Load Leveling

When downstream services cannot handle peak load, use a queue to buffer requests and process them at a sustainable rate.



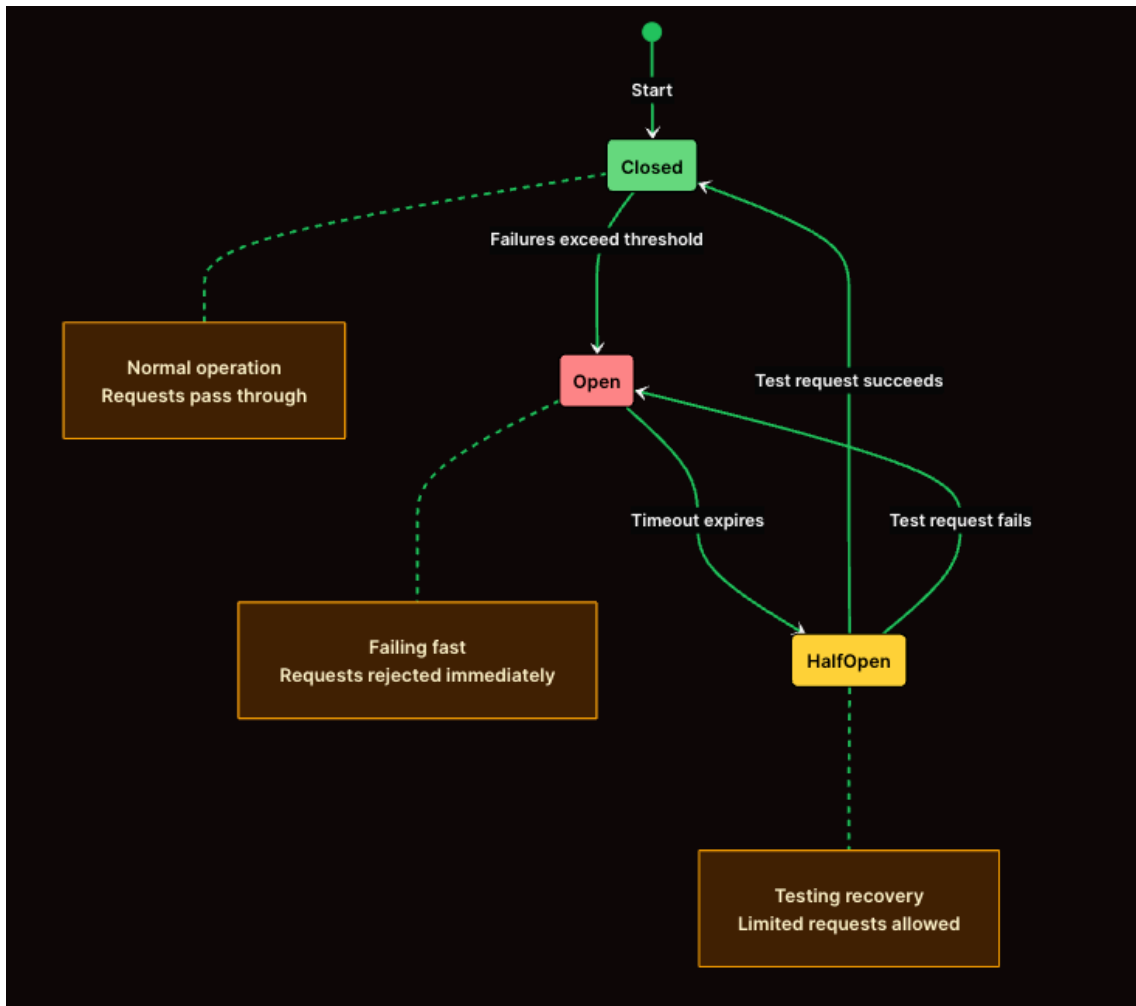
How it provides high availability:

- Decouples producers from consumers allowing them to scale independently
- Buffers traffic spikes that would overwhelm the database

This pattern is essential for handling bursty traffic. A flash sale might generate 100x normal traffic for a few minutes. Without a queue, the database would be overwhelmed. With a queue, orders accumulate and are processed at a sustainable rate.

▼ Pattern 4: Circuit Breaker

When a dependency fails, continuing to call it wastes resources and can cause cascading failures. The circuit breaker pattern prevents this by failing fast.



Circuit Breaker States

State	Behavior	Transitions
Closed	Normal operation. All requests pass through. Track failure rate.	→ Open: when failure rate exceeds threshold
Open	Fail fast. All requests immediately rejected with error.	→ Half-Open: after timeout period
Half-Open	Testing. Allow limited requests through.	→ Closed: if test succeeds / → Open: if test fails

A highly available system stays up even when components fail. But availability alone is not enough. A system that is always up but sometimes gives wrong answers is not trustworthy.

This brings us to our next topic: **reliability**. How do you ensure your system delivers correct results consistently?

▼ 3. Reliability

In the previous chapters, we learned how to scale systems to handle load and how to keep them available when components fail. But there is a third pillar that completes the foundation: reliability.

A reliable system **performs its intended function correctly and consistently**, even in the face of faults. While availability asks "Is the system up?", reliability asks "Is the system doing what it should?"

The distinction matters. Consider a payment system that is always available but occasionally charges customers twice. Or a messaging app that delivers messages out of order. These systems are available, but they are not reliable. And unreliable systems destroy user trust faster than unavailable ones.

What is Reliability?

Reliability is the probability that a system will perform its intended function correctly over a given period of time, under specified conditions.

This definition has several important parts.

- "Correctly" means producing the right output, not just any output.
- "Over a given period" means reliability is measured over time, not at a single instant.
- "Under specified conditions" means we define what normal operation looks like.

An available system responds. A reliable system responds correctly. We want both, but they are distinct properties.

Reliability vs Related Concepts

People often confuse reliability with availability and fault tolerance. Here is how they differ:

Concept	Question It Answers	Example
Availability	Is the system responding?	System returns HTTP 200
Reliability	Is the response correct?	The balance returned is accurate
Fault Tolerance	Does it keep working when components fail?	Works with one database replica down
Durability	Is data preserved despite failures?	Data survives disk failure

A payment system that charges customers twice is available (it processes requests) but unreliable (it processes them incorrectly). A database that loses writes during failover is fault-tolerant (it continues operating) but not durable (data was lost).

▼ Measuring Reliability

You cannot improve what you do not measure. Reliability has several well-established metrics that help you understand how your system behaves over time.

1. Mean Time Between Failures (MTBF)

MTBF measures the average time between failures. A higher MTBF means failures are less frequent.

$$\text{MTBF} = \text{Total Operating Time} / \text{Number of Failures}$$

- System ran for 10,000 hours
- Experienced 5 failures
- $\text{MTBF} = 10,000 / 5 = 2,000$ hours

An MTBF of 2,000 hours means you can expect one failure roughly every 83 days. This helps with planning. If you have 100 servers each with MTBF of 2,000 hours, you will see approximately one server failure per day across your fleet.

Given:

- 100 servers
- Each server has MTBF = 2,000 hours
- We want to find: expected failures per day

Step 1: Calculate the failure rate per server

If MTBF = 2,000 hours, then each server has a failure rate of:

- Failure rate = $1 / \text{MTBF} = 1 / 2,000$ hours

Step 2: Calculate total failure rate across all servers

With 100 servers:

- Total failure rate = $100 \times (1 / 2,000) = 100 / 2,000 = 1 / 20$ failures per hour

Step 3: Convert to failures per day

Since there are 24 hours in a day:

- Failures per day = $(1 / 20) \times 24 = 24 / 20 = 1.2$ failures per day

So approximately **1 to 1.2 failures per day** across the 100-server fleet.

NOTE: The key insight is that with many independent components, their failure rates add up. If one server fails every 2,000 hours, then:

- 10 servers will collectively fail every 200 hours
- 100 servers will collectively fail every 20 hours
- That's roughly once per day

2. Mean Time To Recovery (MTTR)

MTTR measures how long it takes to restore the system after a failure. A lower MTTR means faster recovery.

MTTR = Total Downtime / Number of Failures

- 5 failures occurred
- Total repair time: 10 hours
- $\text{MTTR} = 10 / 5 = 2$ hours per failure

MTTR includes detection time, diagnosis time, repair time, and verification time. Reducing MTTR often has more impact than reducing failure rate. If you cannot prevent failures, at least recover quickly.

3. Error Rate

Percentage of requests that result in errors.

$$\text{Error Rate} = \text{Failed Requests} / \text{Total Requests} \times 100\%$$

System Type	Target Error Rate	Meaning
Critical systems	< 0.01%	1 in 10,000 requests fails
Standard systems	< 0.1%	1 in 1,000 requests fails
Tolerant systems	< 1%	1 in 100 requests fails

4. Data Correctness

Percentage of responses that contain correct data.

$$\text{Correctness} = \text{Correct Responses} / \text{Total Responses} \times 100\%$$

This is the often-overlooked metric. A system can have 99.99% availability and 0.01% error rate, but if 1% of successful responses contain wrong data, you have a reliability problem. Users received a response, it just was not the right one.

▼ Why Systems Become Unreliable

Reliability failures come from multiple sources.

Hardware Failures:

- Physical components wear out and fail. Disks develop bad sectors. Memory corrupts. CPUs overheat. Network cards malfunction.
- The good news is that hardware failures are well-understood and follow predictable statistical patterns. The bad news is that with enough hardware, failures become a daily occurrence.

Software Failures:

- Software bugs are the leading cause of reliability problems. Unlike hardware failures, which tend to be random and independent, software bugs are systematic. Every request that hits the buggy code path fails.
- The most dangerous bugs are those that do not crash the system but silently produce wrong results.

Configuration Errors

- Configuration mistakes are surprisingly common and often catastrophic. A typo in a configuration file can bring down an entire service.
- AWS's S3 outage in 2017 was triggered by a command to remove a small number of servers that accidentally removed a much larger set.

Human Error

- Studies show that human error is the leading cause of outages. Operators mistype commands. Engineers deploy untested changes. On-call responders misdiagnose problems.

Overload and Cascading Failures

- Systems that work perfectly under normal load can fail catastrophically when overloaded. When one component slows down, requests queue up, timeouts fire, retries multiply the load, and the problem cascades.

▼ Key Principles of Reliable Systems

To build reliable systems, engineers typically focus on several core principles:

Redundancy

Redundancy means having backup components ready to take over if one part fails. This could involve multiple servers, duplicate network paths, or backup databases.

Failover Mechanisms

Failover is the process by which a system automatically switches to a redundant or standby component when a failure is detected. This ensures continuous operation without noticeable disruption to users.

Load Balancing

Load balancing distributes incoming traffic across multiple servers. This not only improves performance but also prevents any single server from becoming a single point of failure.

Monitoring and Alerting

A reliable system is constantly monitored. Tools and dashboards track system health and performance, while alerting mechanisms notify engineers of issues before they escalate into major problems.

Graceful Degradation

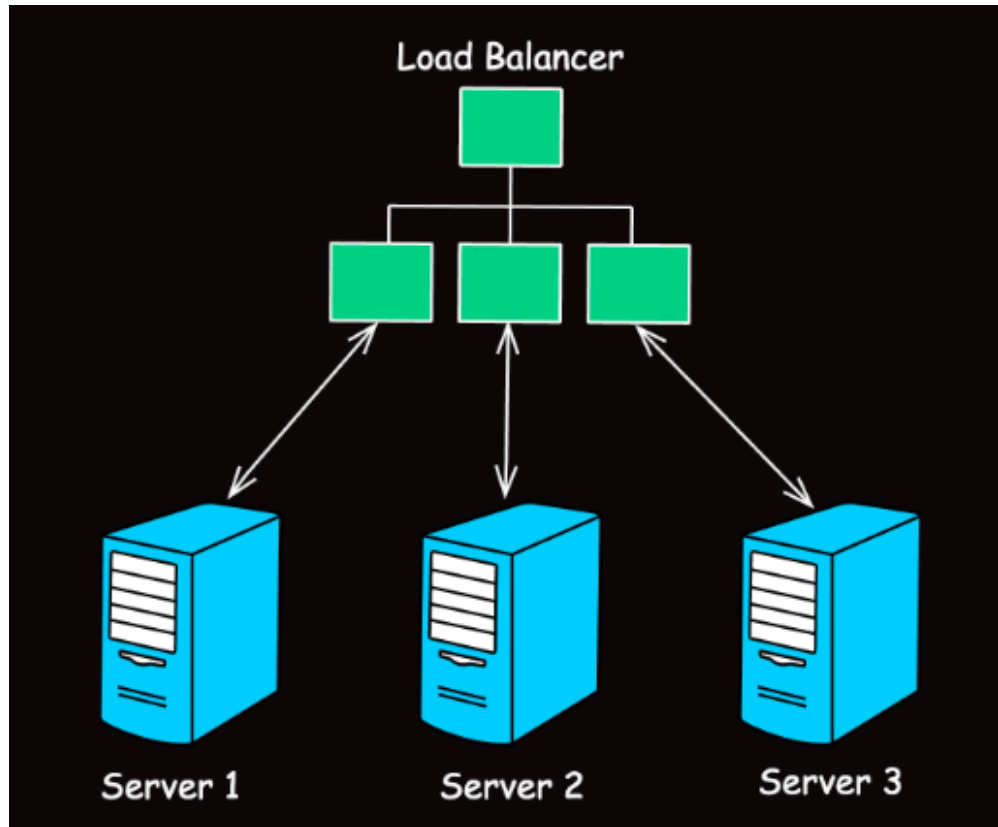
Even when parts of the system fail, a well-designed system can still provide core functionality rather than going completely offline. This concept is known as graceful degradation.

▼ Techniques to Enhance Reliability

1. Redundant Architectures

The most fundamental reliability technique is having more components than you need. If one fails, others continue operating.

For example, if you have a web server handling user requests, deploy several servers behind a load balancer:



If one server fails, the load balancer automatically routes traffic to the remaining servers.

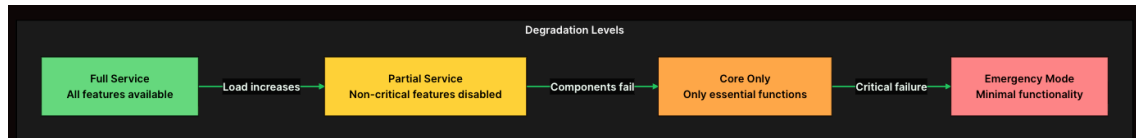
2. Data Replication

Ensure your data is not stored in a single location. Use data replication strategies across multiple databases or data centers.

This way, if one database fails, the system can still access a copy from another location.

3. Graceful Degradation

When parts of the system fail, graceful degradation keeps the core functionality working. Instead of complete failure, the system provides reduced service.

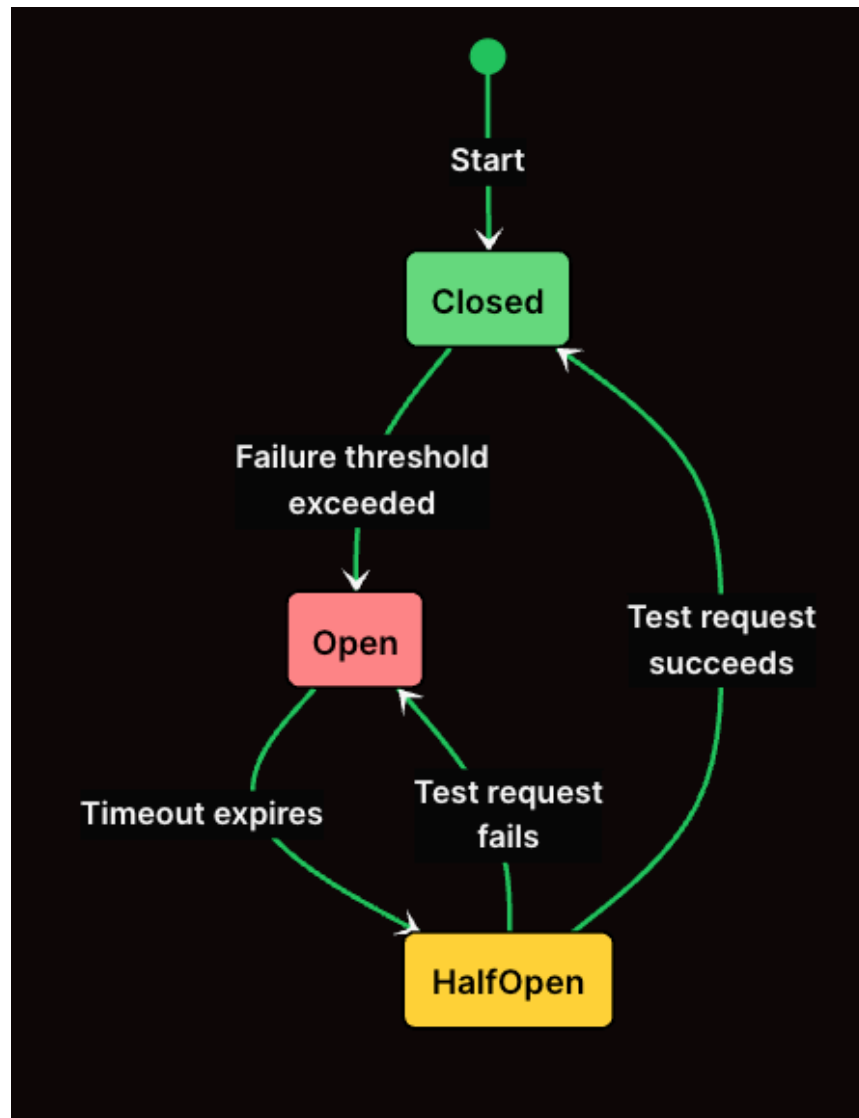


Consider an e-commerce site:

- **Full service:** Personalized recommendations, real-time inventory, all payment options
- **Partial service:** Generic recommendations, cached inventory, primary payment options
- **Core only:** Browse products, checkout with basic payment
- **Emergency mode:** Display cached product pages, accept orders for later processing

4. Circuit Breakers

In a microservices architecture, one service failing can cascade failures throughout the system. Circuit breakers detect when a service is failing and temporarily cut off requests to prevent overload, allowing the system to recover gracefully.



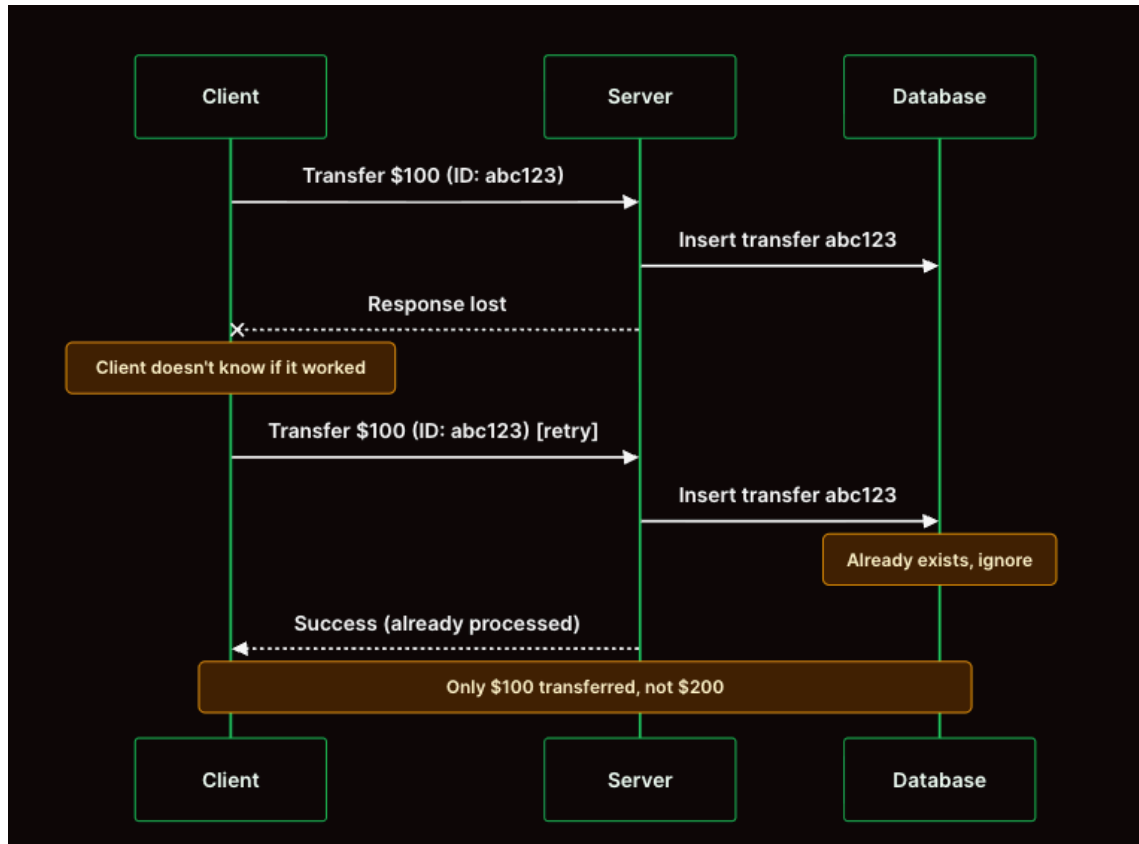
In the **closed** state, requests pass through normally. Failures are counted. When failures exceed a threshold (e.g., 5 failures in 30 seconds), the circuit opens.

In the **open** state, requests fail immediately without calling the dependency. This prevents wasting resources and allows the dependency time to recover.

After a timeout (e.g., 30 seconds), the circuit moves to **half-open**. A limited number of test requests are allowed through. If they succeed, the circuit closes. If they fail, it opens again.

5. Idempotency

Network failures make it unclear whether a request succeeded or failed. If you retry, you might execute the operation twice. Idempotent operations produce the same result regardless of how many times they are executed.



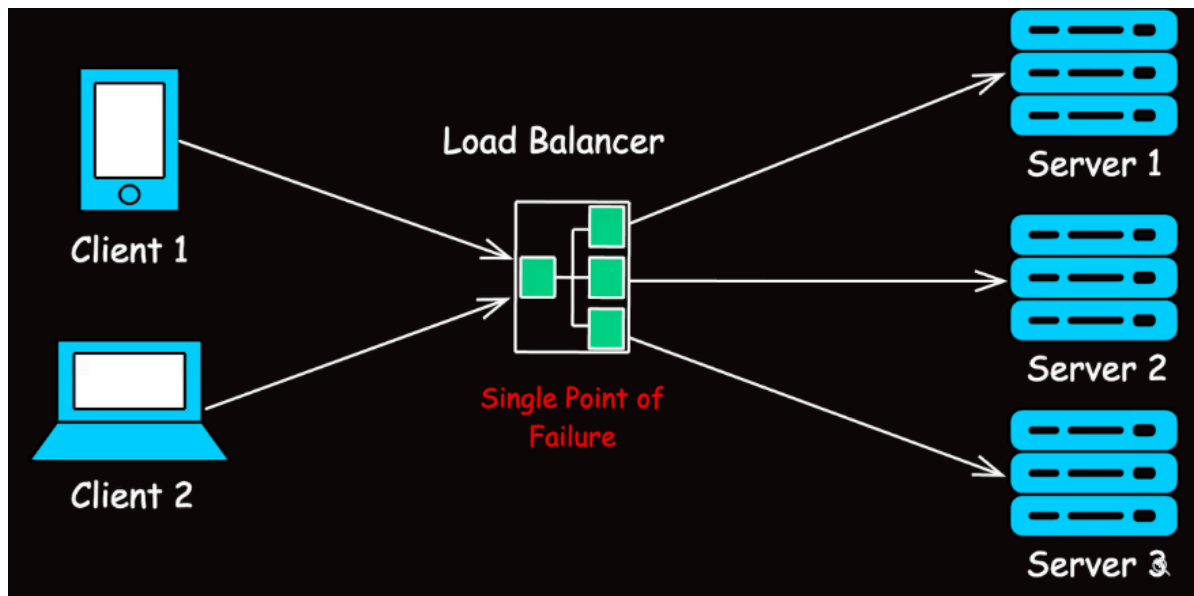
The idempotency key (abc123) allows the server to detect retries. The first execution stores the key. Subsequent executions with the same key return the stored result without re-executing.

Stripe, PayPal, and other payment processors require idempotency keys for money-moving operations. Without them, network issues could cause duplicate charges.

▼ 4. Single Point of Failure (SPOF)

A **Single Point of Failure (SPOF)** is a component in your system whose failure can bring down the entire system, causing downtime, potential data loss, and

unhappy users.



In the above example, if there is only one instance of the **load balancer**, it becomes a SPOF. If it goes down, clients won't be able to communicate with the servers.

By minimizing the number of SPOFs, you can improve the overall **reliability** and **availability** of the system.

In this chapter, we'll explore what a SPOF is, how to identify it in a distributed system, and strategies to avoid it.

▼ 1. Understanding SPOFs

A Single Point of Failure (SPOF) is any component within a system whose failure would cause the entire system to stop functioning.

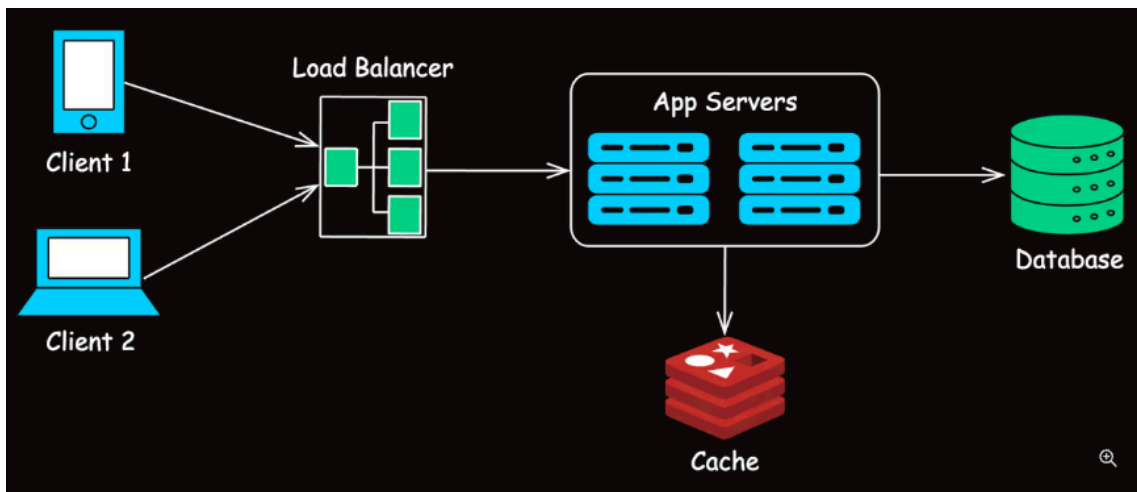
Imagine a bridge that connects two cities. If it's the only route between them and it collapses, the cities are cut off. In this scenario, the bridge is the single point of failure.

In distributed systems, failures are inevitable. Common causes include **hardware malfunctions**, **software bugs**, **power outages**, **network disruptions**, and **human error**.

While failures can't be entirely avoided, the goal is to ensure they don't bring down the entire system.

In system design, SPOFs can include a single server, network link, database, or any component that lacks redundancy or backup.

Let's see an example of a system and various single points of failures in it:



This system has one load balancer, two application servers, one database, and one cache server.

Clients send requests to the load balancer, which distributes traffic across the two application servers. The application servers retrieve data from the cache if it's available, or from the database if it's not.

In this design, the potential SPOFs are:

- **Load Balancer:** If there is only one load balancer instance and it fails, all traffic will stop, preventing clients from reaching the application servers. To avoid this, we can add a **standby** load balancer that can takeover if the primary one fails.
- **Database:** With only one database, its failure would result in data being unavailable, causing downtime and potential data loss. We can avoid this by **replicating** the data across multiple servers and locations.
- **Cache Server:** The cache server is not a true SPOF in the sense that it doesn't bring the entire system down. When it's down, every request hits the database, increasing its load and slowing response times.

The **application servers** are not SPOFs since you have two of them. If one fails, the other can still handle requests, assuming the load balancer can distribute traffic effectively.

▼ 2. How to Identify SPOFs in a Distributed System

1. Map Out the Architecture

Create a detailed diagram of your system's architecture. Identify all components, services, and their dependencies.

Look for components that do not have backups or redundancy.

2. Dependency Analysis

Analyze dependencies between different services and components.

If a single component is required by multiple services and does not have a backup, it is likely a SPOF.

3. Failure Impact Assessment

Assess the impact of failure for each component.

Perform a "**what if**" analysis for each component.

Ask questions like, "What if this component fails?" If the answer is that the system would stop functioning or degrade significantly, then that component is a SPOF.

4. Chaos Testing

Chaos testing, also known as Chaos Engineering, is the practice of intentionally injecting failures and disruptions into a system to understand how it behaves under stress and to ensure it can recover gracefully.

Chaos engineering often involves the use of tools like **Chaos Monkey** (developed by Netflix) that randomly shut down instances or services to observe how the rest of the system responds.

This can help us identify components that, if they fail, would cause a significant impact on the system.

▼ 3. Strategies to Avoid Single Points of Failures

1. Redundancy

The most common way to avoid SPOFs is by adding **redundancy**. Redundancy means having multiple components that can take over if one fails.

Redundant components can be either **active** or **passive**. Active components are always running. Passive (standby) components are only used as a backup when the active component fails.

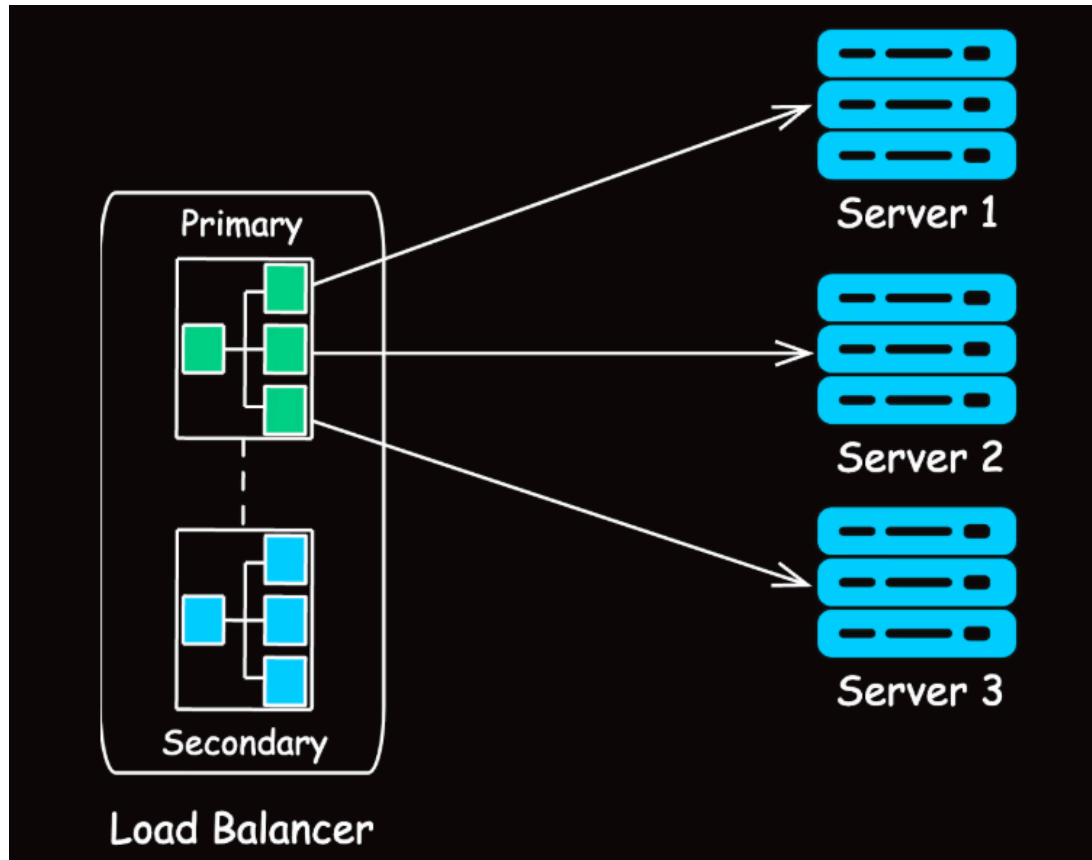


2. Load Balancing

Load balancers distribute incoming traffic across multiple servers, ensuring no single server becomes overwhelmed.

They help avoid single point of failures by detecting failed servers and rerouting traffic to healthy instances.

To prevent the single load balancer becoming a single point of failure, we can add a standby load balancer which can take over if the primary one fails.



3. Data Replication

Data replication involves copying data from one location to another to ensure that data is available even if one location fails.

- **Synchronous Replication:** Data is replicated in real-time to ensure consistency across locations.
- **Asynchronous Replication:** Data is replicated with a delay, which can be more efficient but may result in slight data inconsistencies.

4. Geographic Distribution

Distributing services and data across multiple geographic locations mitigates the risk of regional failures.

This includes using:

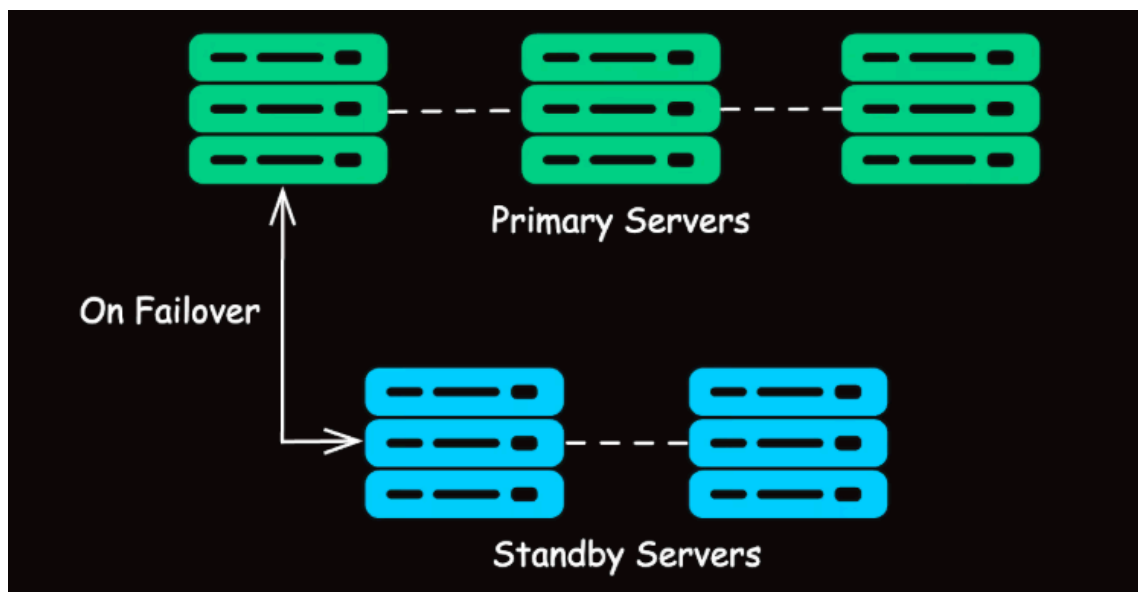
- **Content Delivery Networks (CDNs)** to distribute content globally, improving availability and reducing latency.
- **Multi-Region Cloud Deployments** to ensure that an outage in one region does not disrupt your entire application.

5. Graceful Handling of Failures

Design applications to handle failures without crashing.

Example: If a service that provides user recommendations fails, the application should still function, perhaps with a message indicating limited features temporarily.

Implement failover mechanisms to automatically switch to backup systems when failures are detected.



6. Monitoring and Alerting

Proactive monitoring helps detect failures before they lead to major outages.

Key practices include:

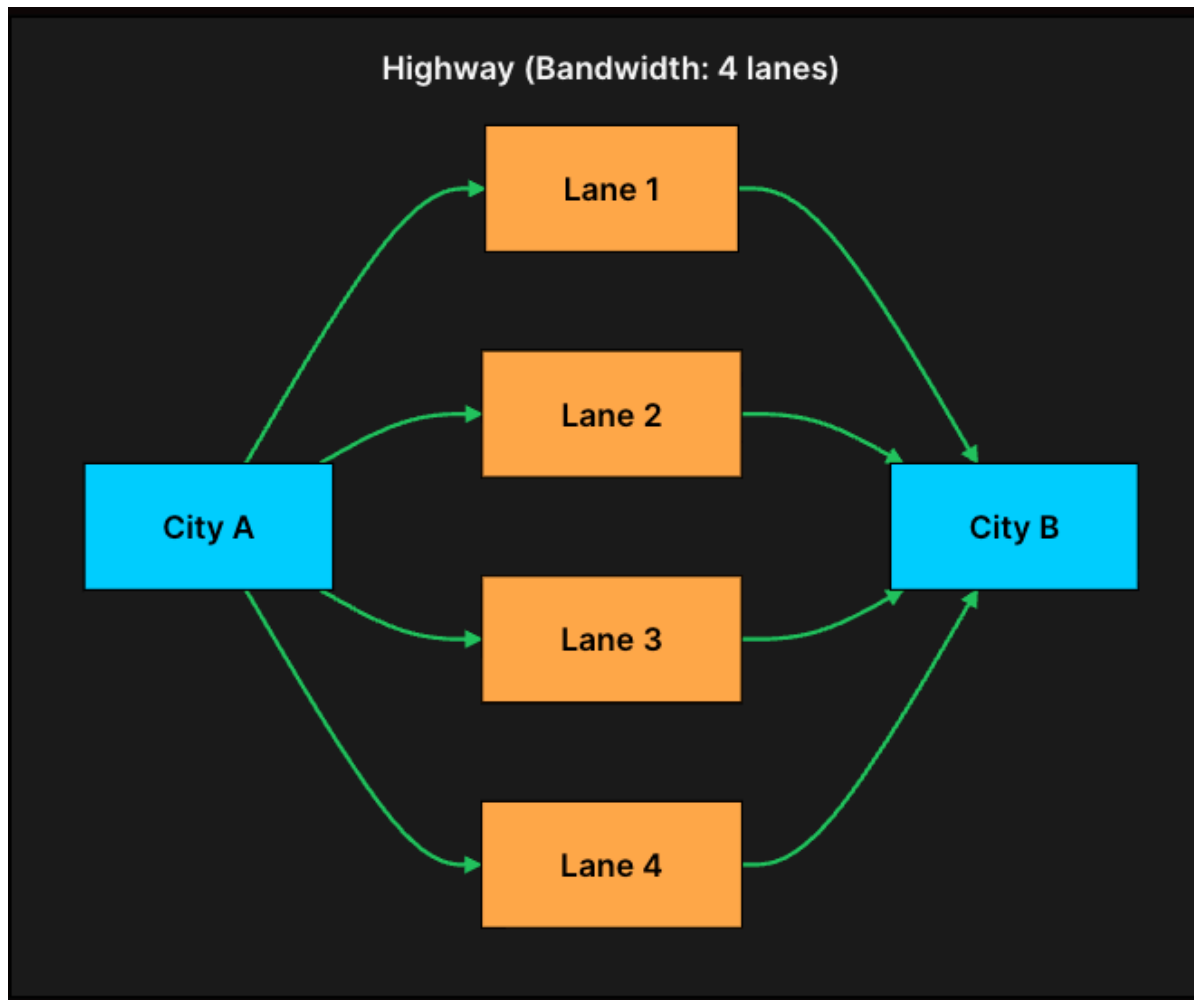
- **Health Checks:** Automated tools that perform regular health checks on components.
- **Automated Alerts:** Alerts and notifications sent when a component fails or behaves abnormally.
- **Self-Healing Systems:** Systems that automatically recover from failures, such as auto-scaling to replace failed servers.

▼ 5. Latency vs Throughput vs Bandwidth

When discussing system performance, three terms come up repeatedly: **latency**, **throughput**, and **bandwidth**. These concepts are often confused or used interchangeably, but they measure fundamentally different things.

The Highway Analogy

Before diving into definitions, let us use a simple analogy. Think of a highway connecting two cities:



- **Bandwidth** is the number of lanes on the highway. More lanes mean more cars can travel simultaneously.
- **Throughput** is how many cars actually pass through per hour. This depends on traffic conditions, not just the number of lanes.
- **Latency** is the time it takes for a single car to travel from one city to the other.

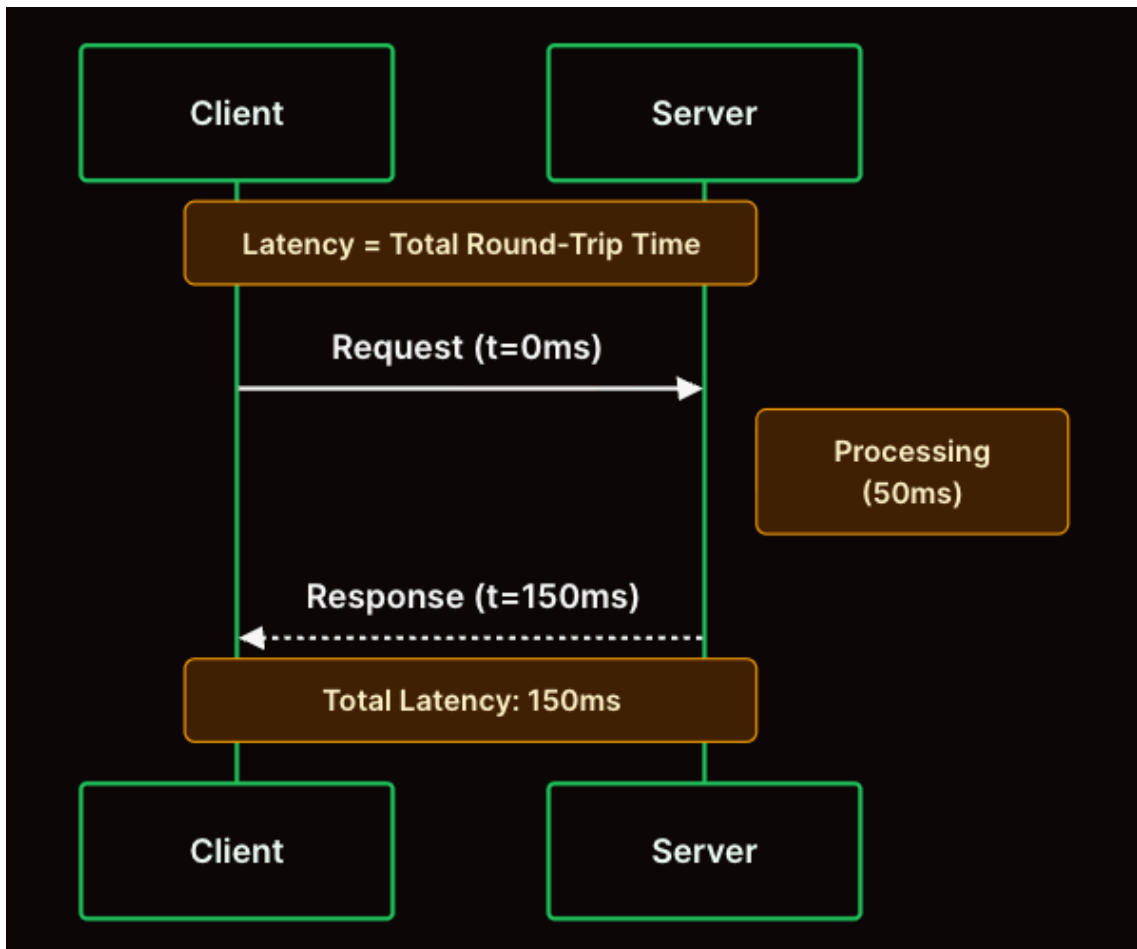
A highway might have 4 lanes (high bandwidth), but if there is an accident, only 100 cars per hour pass through (low throughput). Meanwhile, each car might take 2 hours to complete the journey (high latency).

This analogy helps explain why these metrics do not always move together.

▼ Latency

Latency is the time it takes for a single request to travel from source to destination and back. It measures delay.

In networking, latency is often called **round-trip time (RTT)**, the time from sending a request to receiving a response.



Components of Latency

Latency is not a single value. It is the sum of multiple delays:



1. **Propagation delay:** Time for signals to travel through the medium.
Light in fiber travels at $\sim 200,000$ km/s. A cross-Atlantic request (6,000

km) takes ~30ms just for propagation.

2. **Transmission delay:** Time to push bits onto the wire. Depends on packet size and link bandwidth.
3. **Processing delay:** Time for routers, load balancers, and servers to process packets.
4. **Queuing delay:** Time spent waiting in buffers when components are busy.

Measuring Latency

Latency is typically measured using percentiles:

Metric	Description
p50 (median)	50% of requests are faster than this
p95	95% of requests are faster than this
p99	99% of requests are faster than this
p99.9	99.9% of requests are faster than this

Why percentiles matter: Average latency hides outliers. A system with 10ms average might have p99 of 500ms, meaning 1% of users experience terrible performance.

Example: If an application has a P99 latency of 4ms, it means 99 out of 100 requests finish within 4ms, and only 1 request takes longer than 4ms.

What Affects Latency?

Factor	Impact
Geographic distance	More distance = more propagation delay
Network congestion	Causes queuing delays
Server load	Increases processing time
Database queries	Slow queries add latency

Factor	Impact
DNS resolution	Cold requests need DNS lookup
TLS handshake	Adds 1-2 round trips

Reducing Latency

- **Use CDNs:** Serve content from edge locations closer to users
- **Caching:** Eliminate round trips by caching at multiple layers
- **Connection pooling:** Avoid repeated connection setup
- **Database optimization:** Add indexes, optimize queries
- **Geographic distribution:** Deploy servers closer to users
- **Protocol optimization:** Use HTTP/2, HTTP/3 (QUIC)

▼ Throughput

Throughput is the amount of work completed per unit of time. It measures volume.

For web systems, throughput is often expressed as **requests per second (RPS)** or **transactions per second (TPS)**.

Throughput vs Bandwidth

A common confusion: **bandwidth** is theoretical maximum capacity, while **throughput** is actual achieved rate.

Metric	Definition	Example
Bandwidth	Maximum possible data transfer rate	1 Gbps network link
Throughput	Actual data transfer achieved	600 Mbps actual transfer

You can never have throughput higher than bandwidth, but throughput is almost always lower due to:

- Protocol overhead (headers, acknowledgments)

- Congestion and packet loss
- Processing limitations
- Inefficient resource utilization

Calculating Throughput

For a single-threaded system:

Throughput = 1 / Latency

If latency = 10ms:

Throughput = 1 / 0.01s = 100 requests/second

For a multi-threaded system:

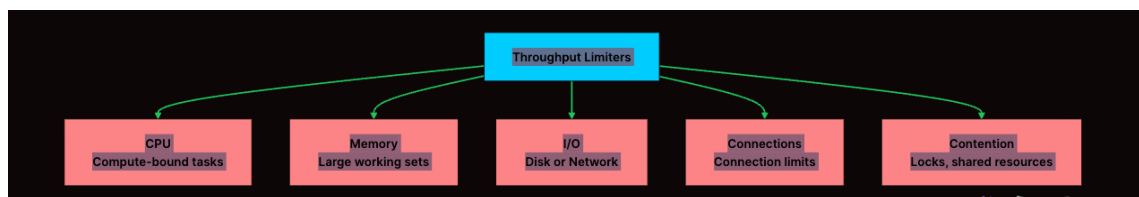
Throughput = Concurrent Workers / Latency

If latency = 10ms and 50 workers:

Throughput = 50 / 0.01s = 5,000 requests/second

What Limits Throughput?

The **bottleneck** determines maximum throughput. A system is only as fast as its slowest component.



Improving Throughput

- **Horizontal scaling:** Add more servers

- **Vertical scaling:** Add more CPU, memory
- **Async processing:** Do not block on slow operations
- **Batching:** Process multiple items together
- **Caching:** Reduce work by reusing results
- **Connection pooling:** Reuse expensive connections
- **Load balancing:** Distribute work evenly

▼ Bandwidth

Bandwidth is the maximum rate at which data can be transferred. It measures capacity.

Bandwidth is typically expressed in **bits per second (bps)**: Kbps, Mbps, Gbps.

Types of Bandwidth

Type	Description
Network bandwidth	Capacity of network links (1 Gbps Ethernet)
Memory bandwidth	Rate of data transfer to/from RAM (DDR4: ~25 GB/s)
Disk bandwidth	Read/write speed of storage (SSD: ~500 MB/s)
Bus bandwidth	Internal data transfer rate (PCIe 4.0: ~64 GB/s)

Bandwidth-Delay Product:

Bandwidth-Delay Product (BDP) = Bandwidth × Latency

BDP represents how much data can be "in flight" at any moment.

- **Bandwidth:** 1 Gbps (Gigabit per second) = (Megabytes per second).
- **Latency (RTT):** 100ms (milliseconds) = 0.1s (seconds).
- **BDP Calculation:** 125 MB/s × 0.1s = 12.5MB

This means 12.5 MB of data can be traveling through the pipe at any instant. If your TCP window size is smaller than BDP, you will not fully

utilize available bandwidth.

In other words, This means that to fully utilize a 1 Gbps pipe over a 100ms path, the sender needs to have 12.5 MB of data acknowledged by the receiver, requiring a window size (buffering) of at least 12.5 MB.

▼ 6. Consistent Hashing

In a **distributed system** where nodes (servers) are frequently **added or removed**, efficiently routing requests becomes challenging.

A common approach is to use **hash the request** and assign it to a server using $\text{Hash}(\text{key}) \bmod N$, where N is the number of servers.

However, this method is highly dependent on the number of servers, and any change in N can lead to **significant rehashing**, causing a major redistribution of keys (requests).

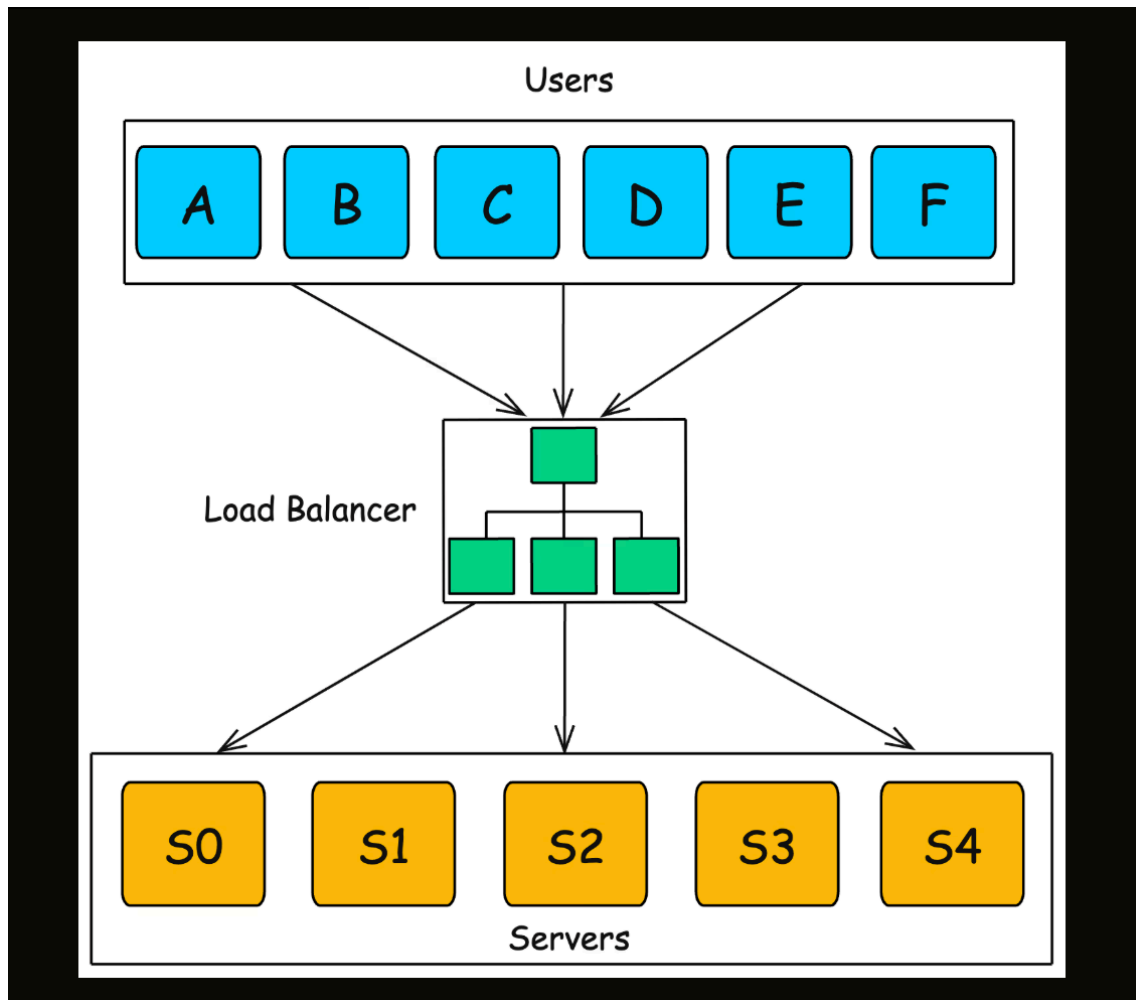
Consistent hashing addresses this issue by ensuring that only a small subset of keys need to be reassigned when nodes are added or removed.

▼ 1. The Problem with Traditional Hashing

Before diving into consistent hashing, let us understand why the naive approach fails so badly.

Imagine you're building a **high-traffic web application** that serves millions of users daily. To handle the load efficiently, you distribute incoming requests across multiple backend servers using a **hash-based load balancer**.

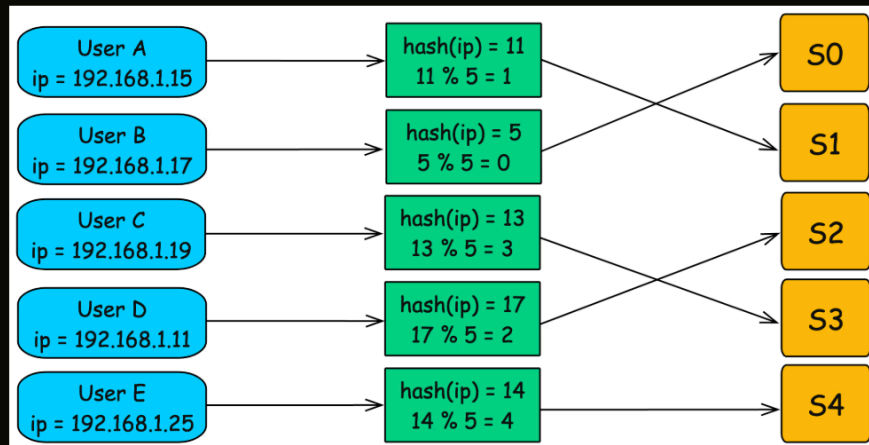
Your system consists of **5 backend servers** (s_0, s_1, s_2, s_3, s_4), and requests are assigned using a hash function that maps each user's **IP address to a specific server**.



The process works like this:

1. The load balancer takes a user's IP address (or session ID).
2. A **hash function** maps the IP to one of the backend servers by taking the **sum of bytes in the IP address** and computing mod 5 (since we have 5 servers).
3. The request is **routed to the assigned server**, ensuring that the same user is always directed to the same server for session consistency.

Example:

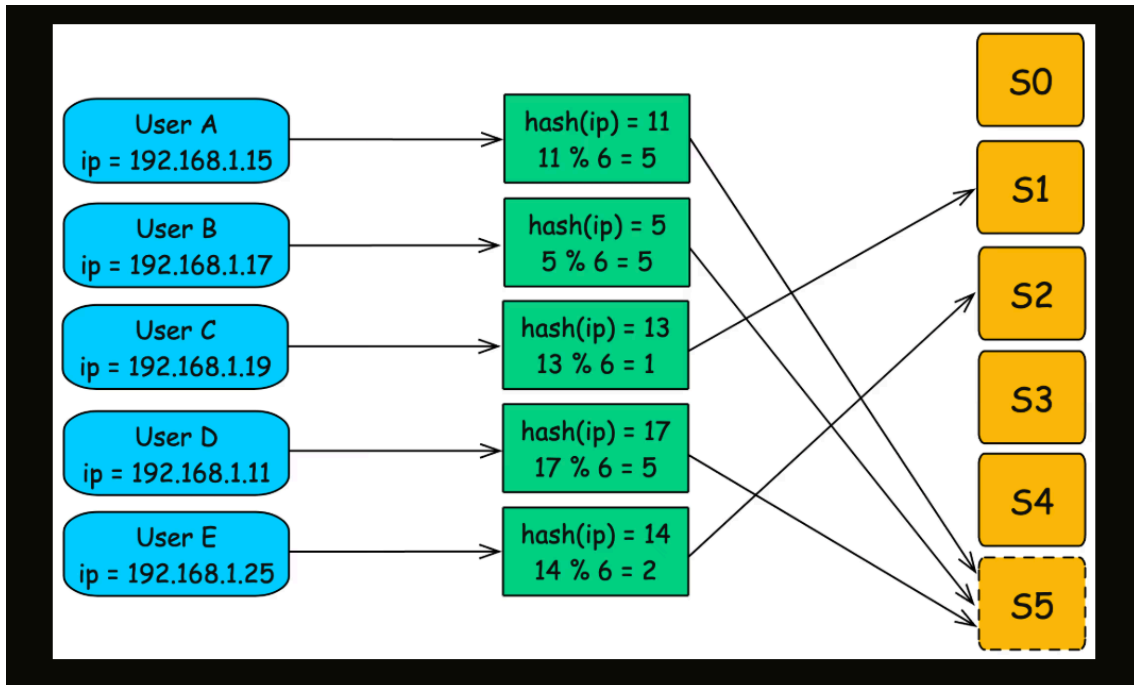


Everything Works Fine... Until You Scale

This approach works **as long as the number of servers remains constant**. But what happens when you **add or remove a server**?

Scenario 1: Adding a New Server (S5)

As traffic increases, you decide to **scale up** by adding a new backend server (**S5**). Now, the hash function must be modified to use **mod 6** instead of **mod 5** since we have 6 servers now.



This seemingly simple change **completely disrupts the existing mapping**, causing **most users to be reassigned to different servers**.

This results into **massive rehashing**, leading to **high overhead**, and **potential downtime**.

Scenario 2: Removing a Server (S4)

Now, let's say one of the servers (**S4**) fails or is removed. The number of servers drops to 4, forcing the hash function to switch from **mod 5** to **mod 4**.

Even though only **one** server was removed, **most users are reassigned to different servers**. This can cause:

- **Session Loss:** Active users may be logged out or disconnected.
- **Cache invalidation:** Cached data becomes irrelevant, increasing database load.
- **Severe performance degradation:** The system may struggle to run efficiently.

The Solution: Consistent Hashing

Consistent hashing offers a more scalable and efficient solution by ensuring that only a **small fraction of users** are reassigned when scaling up or down.

It performs really well when operated in dynamic environments, where the system scales up and down frequently.

▼ 2. How Consistent Hashing Works

Consistent hashing is a **distributed hashing technique** used to efficiently distribute data across multiple nodes (servers, caches, etc.).

It uses a **circular hash space** (hash ring) with a large and constant hash space.

Both nodes (servers, caches, or databases) and keys (data items) are mapped to positions on this hash ring using a **hash function**.

Unlike modulo-based hashing, where changes in the number of nodes cause large-scale remapping, consistent hashing ensures that only a small fraction of keys are reassigned when a node is added or removed, making it highly scalable and efficient.



In consistent hashing, when the number of nodes changes, only k/n keys need to be reassigned, where k is the total number of keys and n is the total number of nodes.

2.1 Constructing the Hash Ring

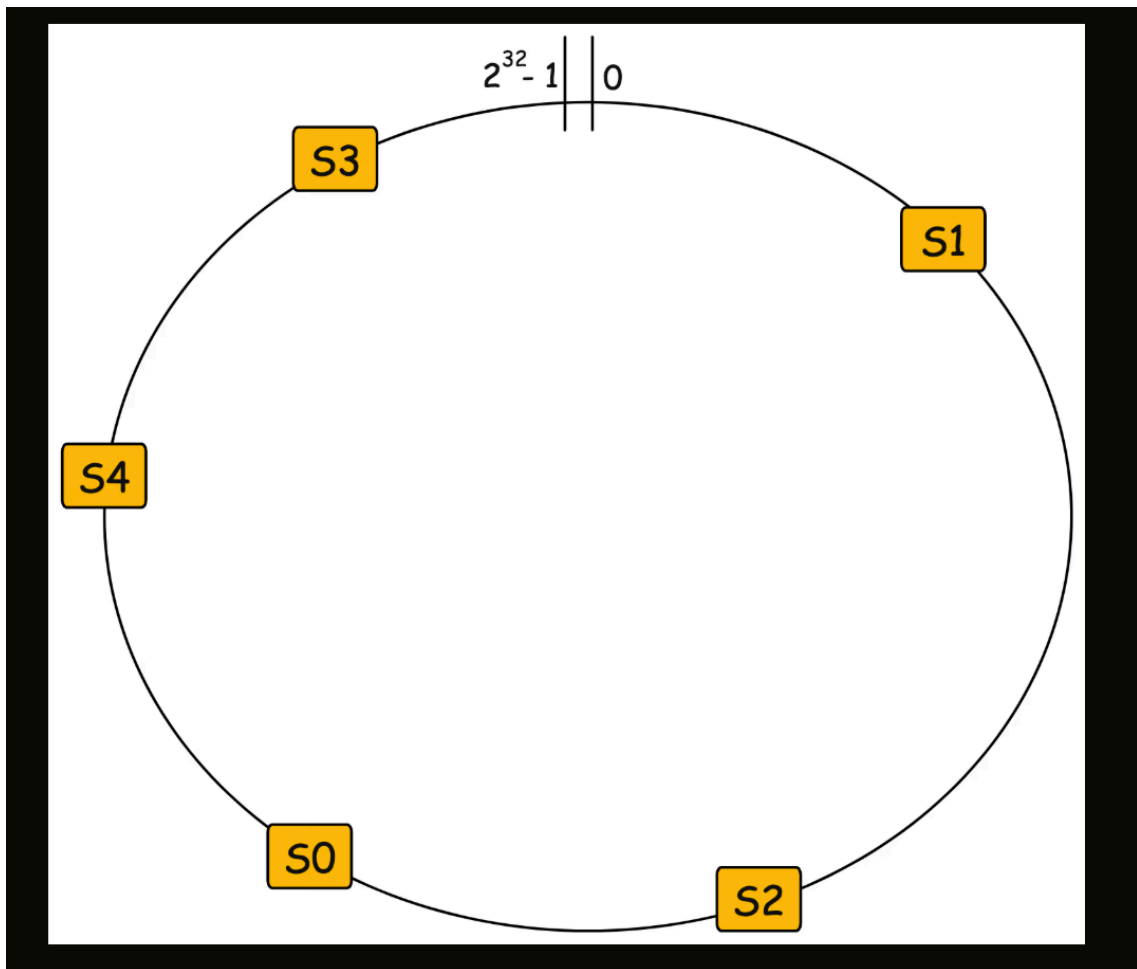
Instead of distributing keys based on $\text{Hash}(\text{key}) \bmod N$, consistent hashing places both servers and keys on a circular hash ring.

Defining the Hash Space

- We use a large, fixed hash space ranging from 0 to $2^{32} - 1$ (assuming a 32-bit hash function).
- This creates a circular structure, where values wrap around after reaching the maximum limit.

Placing Servers on the Ring

- Each **server (node)** is assigned a position on the hash ring by computing `Hash(server_id)`.
- Using the above example with **5 servers** (`S0, S1, S2, S3, S4`), the hash function distributes them at different positions around the ring.



Mapping Keys to Servers

- When a key is added, its position is determined by computing `Hash(key)`.
- Example: a user's request is assigned a position on the ring based on the hash of their IP address: `Hash(IP Address)`

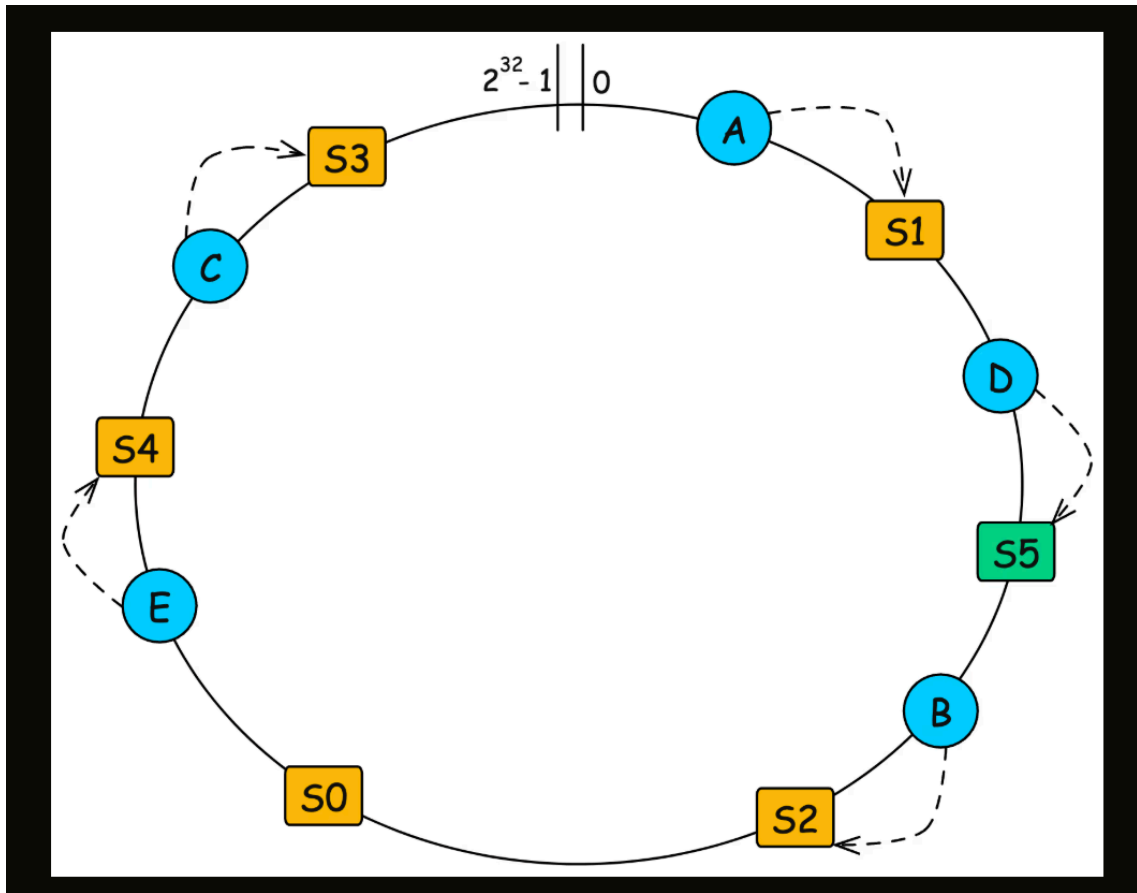
- We then move clockwise around the ring until we find the next available server.
- The key (or request) is assigned to this server for storage or retrieval.

Note: In case a key's hash falls directly on a node's position, it belongs to that node.

2.2 Adding a New Server

Suppose we add a **new server (S5)** to the system.

- The position of S5 falls between S1 and S2 in the hash ring.
- S5 takes over all keys (requests) that fall between S1 and S5, which were previously handled by S2.
- **Example:** User D's requests which were originally assigned to S2, will now be redirected to S5.



This demonstrates how consistent hashing efficiently redistributes keys with minimal disruption, ensuring that only a small subset of keys are reassigned when new servers are added.

2.3 Removing a Node

When a server, such as **S4**, fails or is removed from the system:

- All keys previously assigned to **S4** are reassigned to the next available server in the ring (**S3**).
- Only the keys (requests) that were mapped to **S4** need to move, while all other keys remain unaffected

This results in **minimal data movement**, unlike traditional hashing where removing a node would require reassigning most keys.

▼ 3. Virtual Nodes

In **basic consistent hashing**, each server is assigned **a single position** on the hash ring. However, this can lead to **uneven data distribution**, especially when:

- The number of servers is small.
- Some servers accidentally get clustered together, creating **hot spots**.
- A server is **removed**, causing a sudden load shift to its immediate neighbor.

Virtual nodes (VNodes) are a technique used in consistent hashing to improve **load balancing** and **fault tolerance** by distributing data more evenly across servers.

How Virtual Nodes Work

Instead of assigning one position per server, each physical server is assigned **multiple positions** (virtual nodes) on the hash ring.

- Each server is hashed multiple times to different locations on the ring.
- When a request (key) is hashed, it is assigned to the next virtual node in a clockwise direction.
- The request is then routed to the actual server associated with the virtual node.

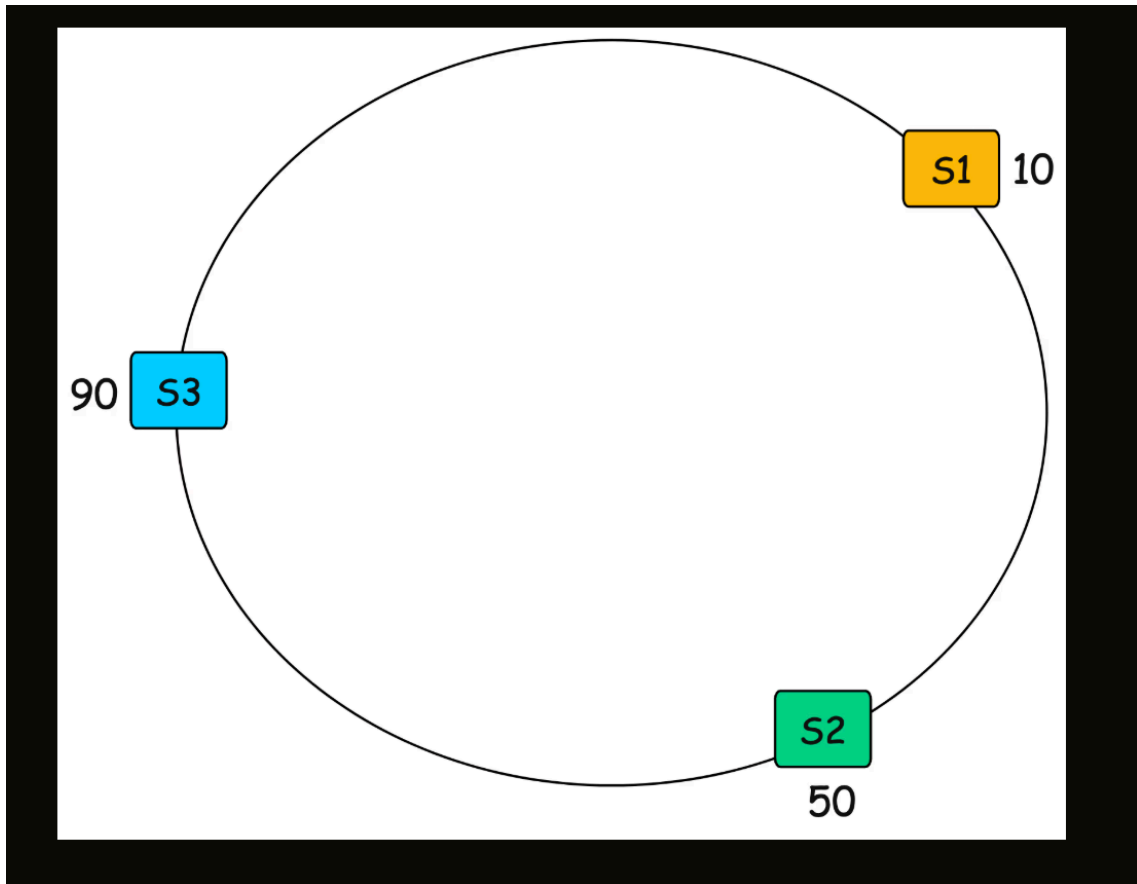
Example

Assume we have **three physical servers (S1, S2, S3)**. Without virtual nodes, their positions might be:

S1 → Position 10

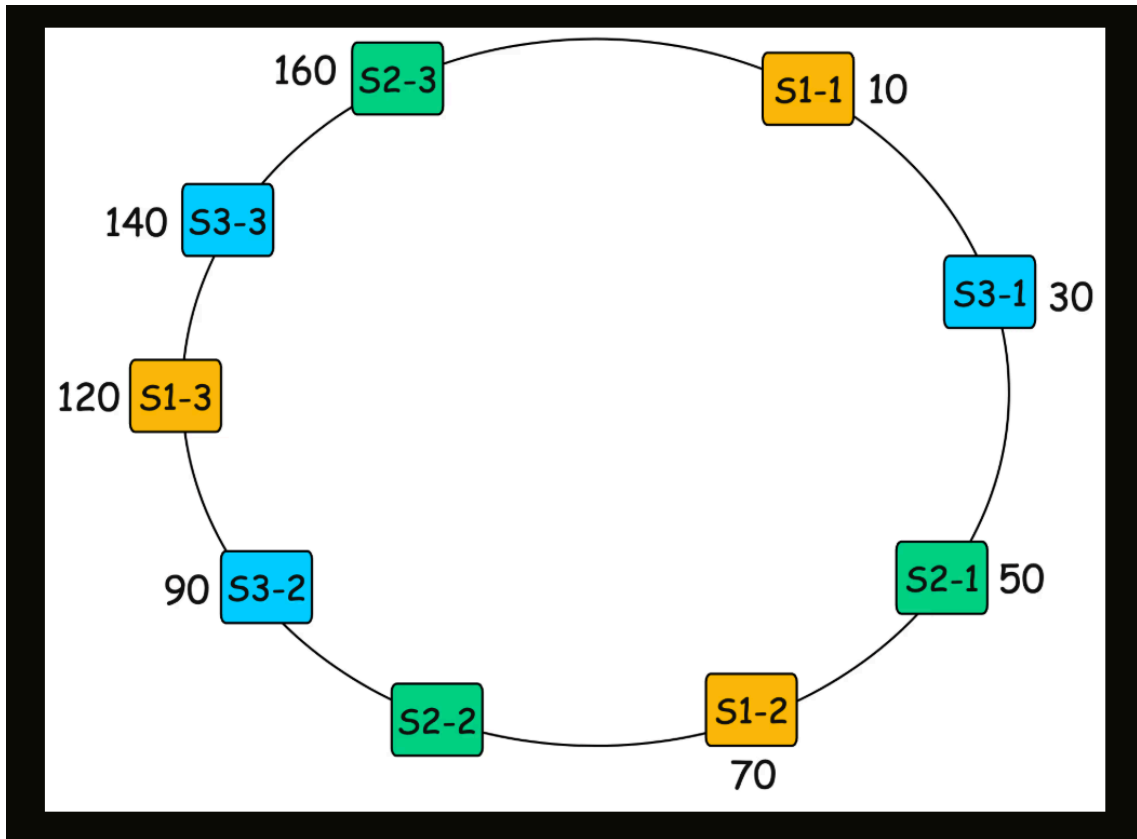
S2 → Position 50

S3 → Position 90



If **S1 fails**, all its keys must be reassigned to **S2**, which can create an overload.

With **virtual nodes**, each server is hashed multiple times:



Now, instead of just one point, **S1** is represented at **multiple positions**, making the distribution **more even**.

If **S1 fails**, its keys are more evenly redistributed among **S2 and S3**, rather than all going to **S2**.

▼ 4. Code Implementation

```
import hashlib
import bisect

class ConsistentHashing:
    def __init__(self, servers, num_replicas=3):
        """
        Initializes the consistent hashing ring.

        - servers: List of initial server names (e.g.,
```

```

["S0", "S1", "S2"])
    - num_replicas: Number of virtual nodes per server for better load balancing
    """
    self.num_replicas = num_replicas # Number of virtual nodes per server
    self.ring = {} # Hash ring storing virtual node mappings
    self.sorted_keys = [] # Sorted list of hash values (positions) on the ring
    self.servers = set() # Set of physical servers (used for tracking)

    # Add each server to the hash ring
    for server in servers:
        self.add_server(server)

def _hash(self, key):
    """Computes a hash value for a given key using MD5."""
    return int(hashlib.md5(key.encode()).hexdigest(), 16)

def add_server(self, server):
    """
    Adds a server to the hash ring along with its virtual nodes.

    - Each virtual node is a different hash of the server ID to distribute load.
    - The server is hashed multiple times and placed at different positions.
    """
    self.servers.add(server)
    for i in range(self.num_replicas): # Creating multiple virtual nodes

```

```

        hash_val = self._hash(f"{server}-{i}") # Unique hash for each virtual node
        self.ring[hash_val] = server # Map hash to the server
        bisect.insort(self.sorted_keys, hash_val)
# Maintain a sorted list for efficient lookup

def remove_server(self, server):
    """
    Removes a server and all its virtual nodes from the hash ring.
    """
    if server in self.servers:
        self.servers.remove(server)
        for i in range(self.num_replicas):
            hash_val = self._hash(f"{server}-{i}")
# Remove each virtual node's hash
            self.ring.pop(hash_val, None) # Delete from hash ring
            self.sorted_keys.remove(hash_val) # Remove from sorted key list

def get_server(self, key):
    """
    Finds the closest server for a given key.

    - Hash the key to get its position on the ring.
    - Move clockwise to find the nearest server.
    - If it exceeds the last node, wrap around to the first node.
    """
    if not self.ring:
        return None # No servers available

    hash_val = self._hash(key) # Hash the key
    index = bisect.bisect(self.sorted_keys, hash_val)

```

```

l) % len(self.sorted_keys) # Locate nearest server
    return self.ring[self.sorted_keys[index]] # Re
turn the assigned server

# ----- Usage Example -----

# Step 1: Initialize Consistent Hashing with servers
servers = ["S0", "S1", "S2", "S3", "S4", "S5"]
ch = ConsistentHashing(servers)

# Step 2: Assign requests (keys) to servers
print("UserA is assigned to: " + ch.get_server("User
A")) # Maps UserA to a server
print("UserB is assigned to: " + ch.get_server("User
B")) # Maps UserB to a server

# Step 3: Add a new server dynamically
ch.add_server("S6")
print("UserA is now assigned to: " + ch.get_server("Use
rA")) # Might be reassigned if affected

# Step 4: Remove a server dynamically
ch.remove_server("S2")
print("UserB is now assigned to: " + ch.get_server("Use
rB")) # Might be reassigned if affected

```

Explanation:

Key Components

- **Hash Ring (`self.ring`)**: Stores hash values → server mappings. Uses **virtual nodes** (replicas) for better load balancing.
- **Sorted Keys (`self.sorted_keys`)**: Maintains a **sorted list** of hash values for efficient lookups.

- **Server Set (`self.servers`)**: Tracks active physical servers.

Initialization (`__init__`)

- Calls `add_server()` for each server, hashing it multiple times (based on `num_replicas`) to ensure even distribution.

Hashing Function (`_hash`)

- Uses **MD5 hashing** to convert strings into large integers for consistent placement on the hash ring.

Adding a Server (`add_server`)

- Generates multiple hash values for each server (`server-0` , `server-1` , etc.).
- Stores these in `self.ring` and maintains a sorted order in `self.sorted_keys` for fast lookup.

Removing a Server (`remove_server`)

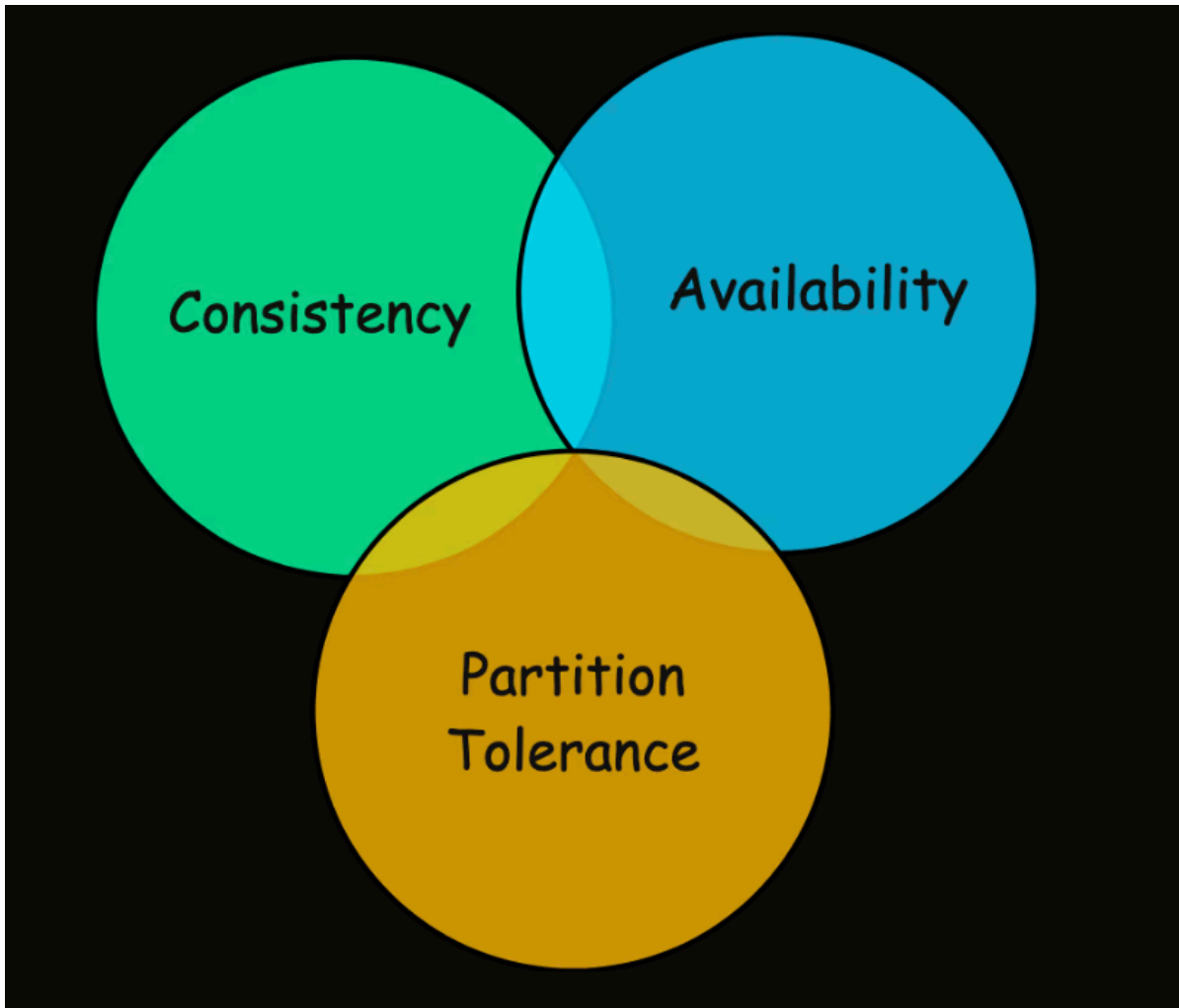
- Deletes the server's hash values and its virtual nodes from `self.ring` and `self.sorted_keys` .

Getting a Server (`get_server`)

- Hashes the input key and finds the closest **clockwise** server using `bisect.bisect()` . Wraps around to the first node if necessary.

▼ 7. CAP Theorem

The **CAP theorem** provides a fundamental framework for understanding the **trade-offs** that must be made when designing distributed systems.



CAP stands for **Consistency**, **Availability**, and **Partition Tolerance**, and the theorem states that:

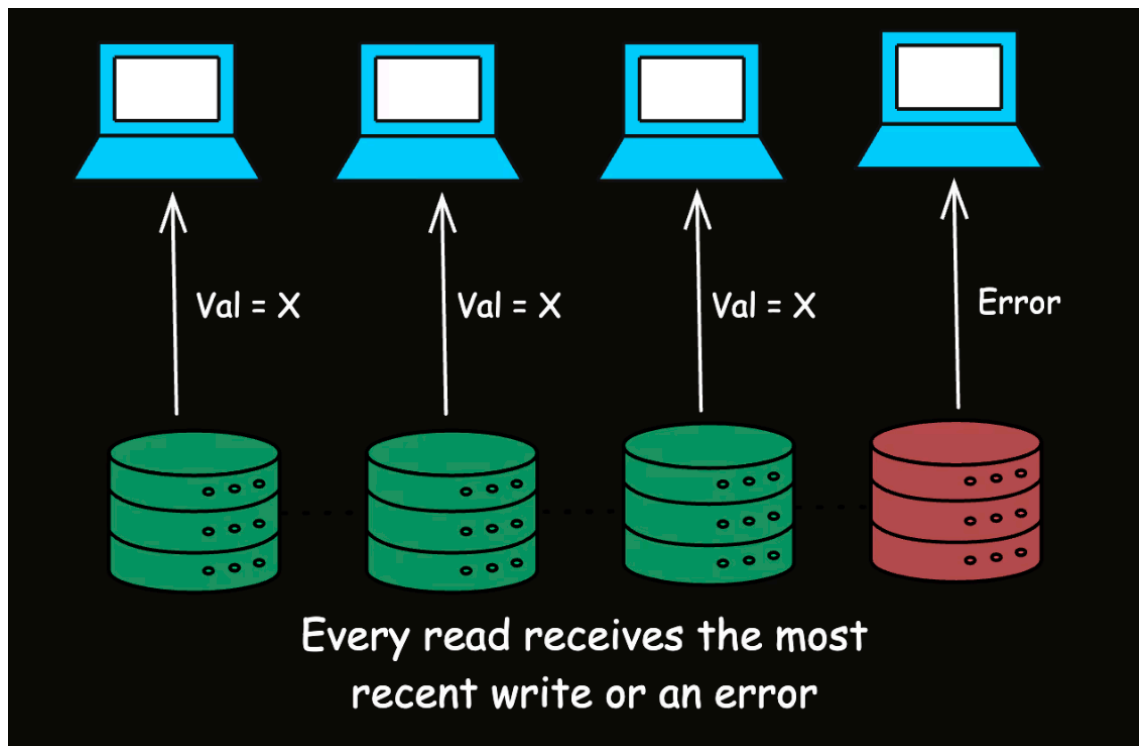
It is impossible for a distributed data store to simultaneously provide all three guarantees.

- **Consistency (C):** Every read receives the most recent write or an error.
- **Availability (A):** Every request (read or write) receives a non-error response, without guarantee that it contains the most recent write.
- **Partition Tolerance (P):** The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.

▼ 1. Three Pillars of CAP

1. Consistency

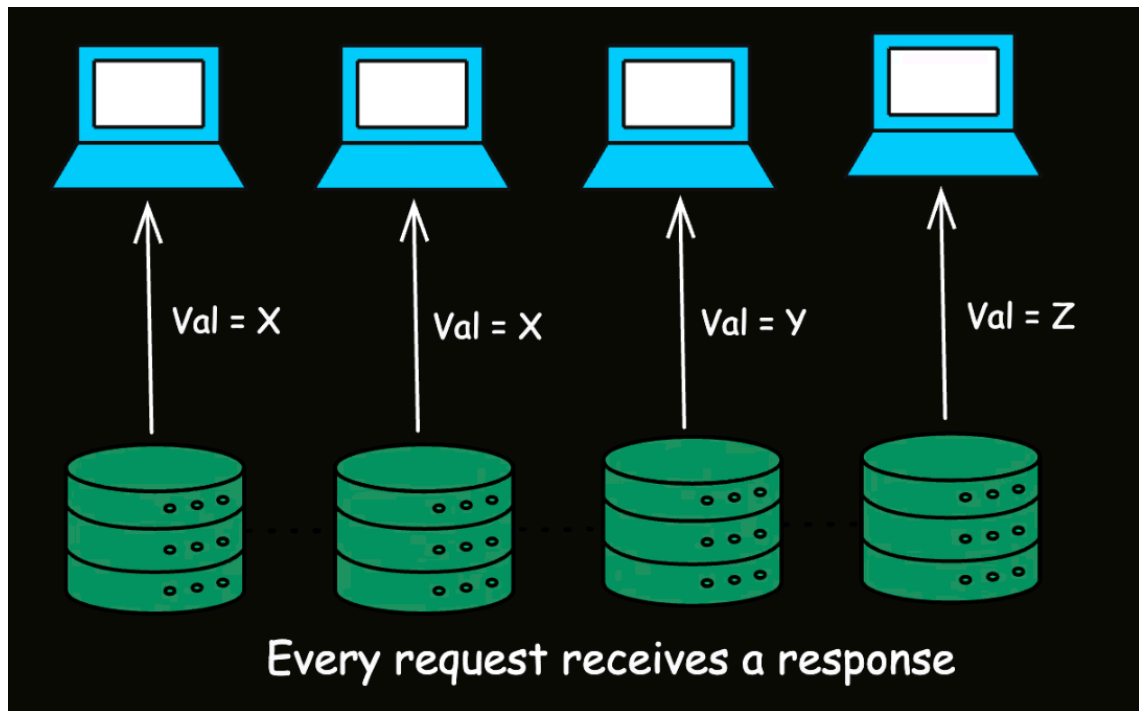
Consistency ensures that **every read receives the most recent write or an error**. This means that all working nodes in a distributed system will return the same data at any given time.



Consistency is crucial for applications where having the most up-to-date data is critical, such as financial systems, where a balance inquiry must reflect the most up-to-date state of an account.

2. Availability

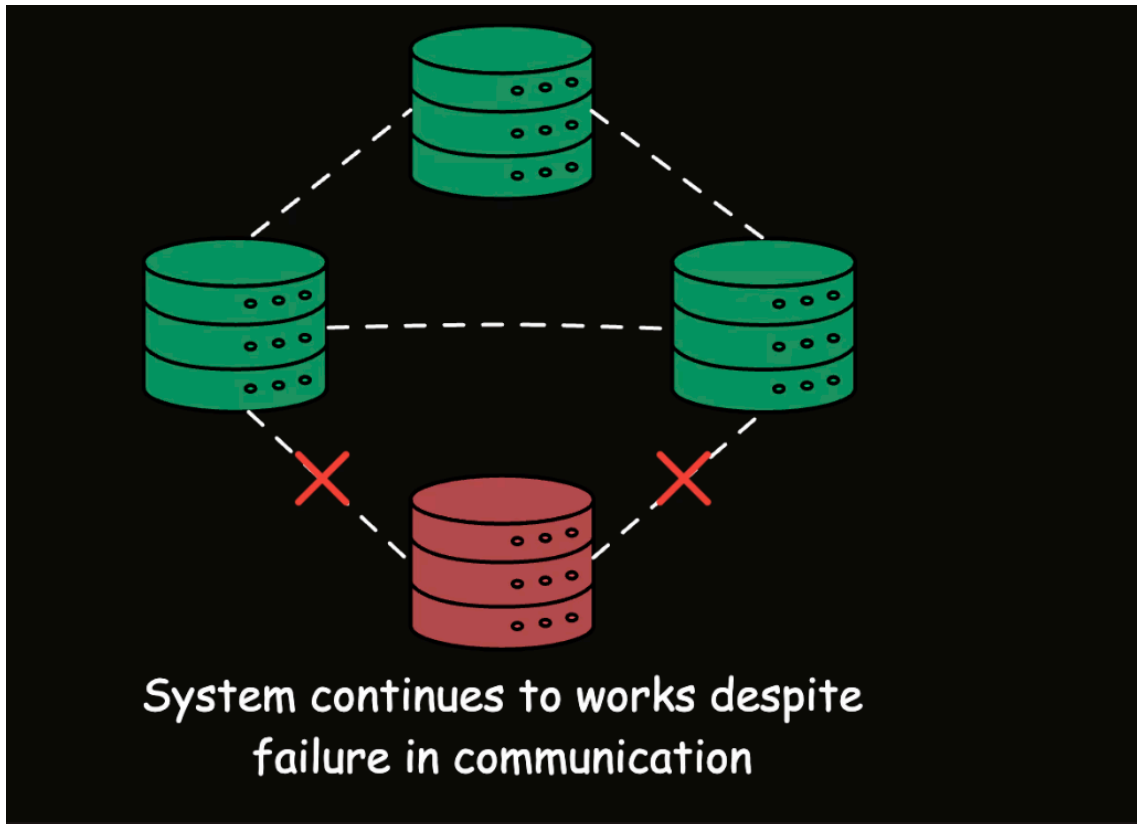
Availability guarantees that **every request (read or write) receives a response**, without ensuring that it contains the most recent write. This means that the system remains **operational** and **responsive**, even if the response from some of the nodes don't reflect most up-to-date data.



Availability is important for applications that need to remain operational at all times, such as online retail systems.

3. Partition Tolerance

Partition Tolerance means that the **system continues to function despite network partitions** where nodes cannot communicate with each other.



A **network partition** occurs when a network failure causes a distributed system to split into two or more groups of nodes that cannot communicate with each other.

When there is a network partition, the system must choose between **Consistency** and **Availability**.

Partition Tolerance is essential for distributed systems because network failures can and do happen. A system that tolerates partitions can maintain operations across different network segments.

▼ 2. The CAP Trade-Off: Choosing 2 out of 3

The CAP theorem asserts that in the presence of a network partition, a distributed system must choose between **Consistency** and **Availability**.

Let's explore these scenarios:

CP (Consistency and Partition Tolerance)

These systems prioritize consistency and can tolerate network partitions, but at the cost of availability. During a partition, the system may reject some requests to maintain consistency.

Traditional relational databases, such as MySQL and PostgreSQL, when configured for strong consistency, prioritize consistency over availability during network partitions.

Banking systems typically prioritize **consistency over availability** since data accuracy is more critical than availability during network issues.

AP (Availability and Partition Tolerance)

These systems ensure availability and can tolerate network partitions, but at the cost of consistency. During a partition, different nodes may return different values for the same data.

NoSQL databases like Cassandra and DynamoDB are designed to be highly available and partition-tolerant, potentially at the cost of strong consistency.

Amazon's shopping cart system is designed to always accept items, prioritizing availability. When you add items to your Amazon cart, the action almost never fails, even during high traffic periods like Black Friday.

CA (Consistency and Availability)

In the absence of partitions, a system can be both consistent and available. However, network partitions are inevitable in distributed systems, making this combination impractical.

Example Systems: Single-node databases can provide both consistency and availability but aren't partition-tolerant. In a distributed

| setting, this combination is theoretically impossible.

▼ 3. Practical Design Strategies

Designing distributed systems requires carefully balancing these trade-offs based on application requirements.

Here are some practical strategies:

1. Eventual Consistency

For many systems, strict consistency isn't always necessary.

It's a version of consistency model used in distributed systems, where all updates made to a data store will eventually be propagated to all replicas or nodes in the system, resulting in a consistent state over time.

It is a weak form of consistency that allows for temporary inconsistencies or conflicts in the data until all updates are fully propagated across all nodes.

| **Example:** Systems where immediate consistency is not critical, such as DNS and content delivery networks (CDNs).

2. Strong Consistency

A model ensuring that once a write is confirmed, any subsequent reads will return that value.

| **Example:** Systems requiring high data accuracy, like financial applications and inventory management.

3. Tunable Consistency

Tunable consistency allows systems to adjust their consistency levels based on specific needs, providing a balance between strong and eventual consistency.

Systems like Cassandra allow configuring the level of consistency on a per-query basis, providing flexibility.

Example: Applications needing different consistency levels for different operations, such as e-commerce platforms where order processing requires strong consistency but product recommendations can tolerate eventual consistency.

4. Quorum-Based Approaches:

Quorum-based approaches use voting among a group of nodes to ensure a certain level of consistency and fault tolerance.

Example: Systems needing a balance between consistency and availability, often used in consensus algorithms like Paxos and Raft.

▼ 4. Beyond CAP: PACELC

While CAP is foundational, it doesn't cover all scenarios.

Daniel Abadi proposed the **PACELC** theorem as an extension by introducing **latency** and **consistency** as additional attributes of distributed systems.

- If there is a partition (P), the trade-off is between availability and consistency (A and C).
- Else (E), the trade-off is between **latency (L)** and **consistency (C)**.

This theorem acknowledges that even when the system is running normally, there's a tradeoff between latency and consistency.

In conclusion, the CAP Theorem is a powerful tool for understanding the inherent trade-offs in distributed system design. It's not about choosing the "best" property, but rather about making informed decisions based on the specific needs of your application.

By carefully evaluating the CAP trade-offs, you can architect robust and resilient systems that deliver the right balance of consistency, availability, and partition tolerance.

5. Why consistency and availability cannot coexist in distributed systems?

While it may be achievable in a single server mode, it could be considered a single point of failure, therefore making availability uncertain. In distributed systems, with multiple partitions, attaining both consistency and availability is not feasible.

▼ 8. Failover

Failover is the ability to switch automatically and seamlessly to a reliable backup system. When a component or primary system fails, either a standby operational mode or redundancy should achieve failover and lessen or eliminate negative impact on users.

Types of Failover Configurations: Active-Active vs Active-Passive

Active-active and active-passive or active-standby are the most common configurations for high availability (HA). Each implementation technique achieves failover in a different way, although both improve reliability.

Typically, at least two nodes actively and simultaneously running the same sort of service comprise an active-active high availability cluster. The active-active cluster distributes workloads across all the nodes more evenly, preventing any single node from overloading and achieving load balancing. And because more nodes remain available, throughput and response times improve. To ensure the HA cluster operates seamlessly and achieves redundancy, the individual configurations and settings of the nodes should be identical.

In contrast, in an active-passive cluster, although there must be at least two nodes, not all of them are active. In a two node system with the first node active, the second node will remain passive or on standby as the failover server. In this standby operational mode, it can remain ready should the active,

primary server stop functioning to serve as a backup. However, unless there is a failure, clients only connect to the active server.

Just as in the active-active cluster, both servers in the active-standby cluster must be configured with the very same settings. This way, clients cannot perceive any change in service, even if the failover router or server must take over.

Clearly, in an active-standby cluster although the standby node is always running, actual utilization approaches zero.

In an active-active cluster, utilization of both nodes nears half and half—although each node can handle the entire load alone. However, this also means that node failure can cause performance to degrade if one active-active configuration node handles more than half of the load consistently.

Outage time during a failure is virtually zero with an active-active HA configuration, because both paths are active. With an active-passive configuration, outage time has the potential to be greater, as the system must switch from one node to the other, which requires time.

Active-Active Configuration

- Multiple servers (nodes) run the same service simultaneously.
- All nodes are active and share the workload evenly.

Benefits:

- Load balancing: Distributes workloads across all nodes, preventing overload on any single server.
- High availability: If one node fails, others continue operating without interruption.
- Improved performance: Handles requests faster with higher throughput.

Requirements:

- Servers must have identical configurations for consistency.

Considerations:

- If a node fails, remaining nodes handle increased load until recovery.

Active-Passive Configuration

- One server (active node) handles all operations; another server (passive node) stays on standby.
- The passive node is synchronized and ready to take over if the active server fails.

Benefits:

- Standby backup: Passive server is ready but not handling traffic under normal conditions.
- Simpler failover: Switches to passive node when failure occurs.
- Lower resource use: Passive server remains mostly idle, saving costs.

Considerations:

- Failover can involve a short delay while the passive server takes control.
- Overall throughput is lower since only one server is active at a time.

▼ 9. Fault Tolerance

November 25, 2020. If you work in tech infrastructure, that's a date you probably remember. On that day, AWS's US-east-1 experienced a significant outage, and it broke a pretty significant percentage of the internet.

Adobe, League of Legends, Roku, Sirius XM, Amazon, Flickr, Giphy, and many, many more experienced issues or went offline completely as a result of the outage.

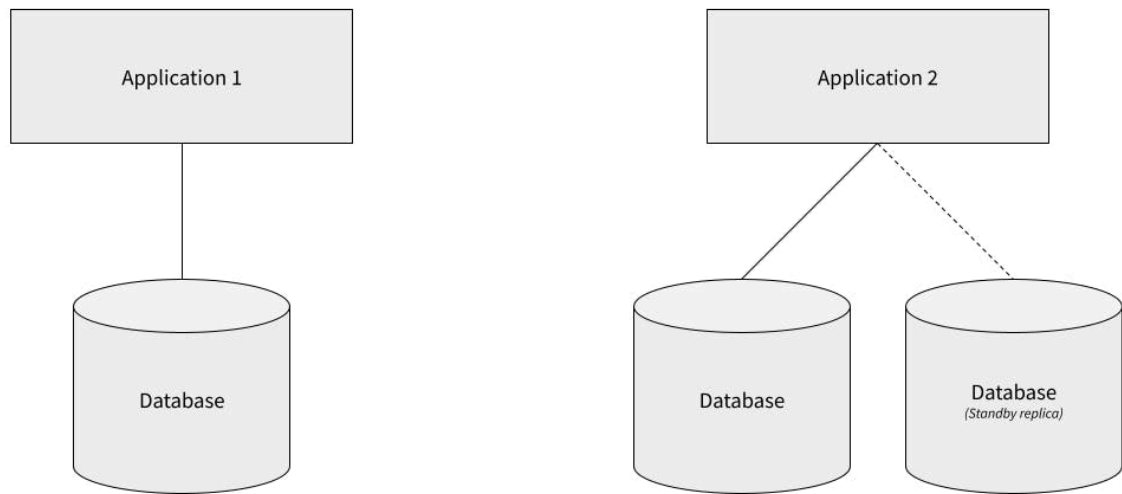
That kind of outage costs time and money. It also does something that's arguably even more expensive in the long run: it erodes customer confidence in your product.

Outages like that are one of the reasons why fault tolerance is an integral part of most modern application architectures.

▼ What is fault tolerance?

Fault tolerance describes a system's ability to handle errors and outages without any loss of functionality.

For example, here's a simple demonstration of comparative fault tolerance in the database layer. In the diagram below, **Application 1** is connected to a single database instance. **Application 2** is connected to two database instances — the primary database and a standby replica.



In this scenario, Application 2 is more fault tolerant. If its primary database goes offline, it can switch over to the standby replica and continue operating as usual.

Application 1 is not fault tolerant. If its database goes offline, all application features that require access to the database will cease to function.

Of course, this is just a simple example. In reality, fault tolerance must be considered in every layer of a system (not just the database), and there are degrees of fault tolerance.

▼ **Fault tolerance can also be achieved in a variety of ways. These are some of the most common approaches to achieving fault tolerance:**

- **Multiple hardware systems** capable of doing the same work. For example, Application 2 in our diagram above could have its two databases located on two different physical servers, potentially in different locations. That way, if the primary database server

experiences an error, a hardware failure, or a power outage, the other server might not be affected.

- **Multiple instances of software** capable of doing the same work. For example, many modern applications make use of containerization platforms such as Kubernetes so that they can run multiple instances of software services. One reason for this is so that if one instance encounters an error or goes offline, traffic can be routed to other instances to maintain application functionality.
- **Backup sources of power**, such as generators, are often used in on-premises systems to protect the application from being knocked offline if power to the servers is impacted by, for example, the weather. That type of outage is more common than you might expect.

▼ High Availability vs. Fault Tolerance

What is high availability? **High availability** refers to a system's total uptime, and achieving high availability is one of the primary reasons architects look to build fault-tolerant systems.

Technically, when discussing fault tolerance vs high availability, the two are not exactly the same thing. Keeping an application highly available is not simply a matter of making it fault tolerant. A highly fault-tolerant application could still fail to achieve high availability if, for example, it has to be taken offline regularly to upgrade software components, change the database schema, etc.

In practice, however, the two are often closely connected, and it's difficult to achieve high availability – whether it's for high availability servers or high availability clusters – without robust, fault-tolerant systems.

▼ Fault tolerance goals

Building fault-tolerant systems is more complex and generally also more expensive. If we think back to our simple example from earlier, Application 2 is more fault tolerant, but it also has to pay for and maintain an additional database server. Thus, it's important to assess the level of fault tolerance your application requires and build your system accordingly.

Normal functioning vs. graceful degradation

When designing fault-tolerant systems, you may want the application to remain online and fully functional at all times. In this case, your goal is **normal functioning** — you want your application, and by extension the user's experience, to remain unchanged even if an element of your system fails or is knocked offline.

Another approach is aiming for what's called **graceful degradation**, where outages and errors are allowed to impact functionality and degrade the user experience, but not knock the application out entirely. For example, if a software instance encounters an error during a period of heavy traffic, the application experience may slow for other users, and certain features might become unavailable.

Building for normal functioning obviously provides for a superior user experience, but it's also generally more expensive. Mission-critical applications and systems will likely need to maintain normal functioning in all but the most dire of disasters, whereas it might make economic sense to allow less essential systems to degrade gracefully.

▼ Setting survival goals

Achieving 100% fault tolerance isn't really possible, so the question architects generally have to answer when designing fault-tolerant systems is how much they want to be able to survive.

Survival goals can vary, but here are some common ones for applications that run on one or more of the public clouds, in ascending order of resilience:

- **Survive node failure.** Running instances of your software on multiple nodes (often different physical servers) with the same AZ (data center) can allow your application to survive node level failures.
- **Survive AZ failure.** Running instances of your software across multiple availability zones (data centers) within a cloud region will allow you to survive AZ outages, such as a specific data center losing power during a storm.

- **Survive region failure.** Running instances of your software across multiple cloud regions can allow you to survive an outage affecting an entire region, such as the AWS US-east-1 outage.

Architects also need to consider factors such as RTO and RPO to minimize the negative impact when outages do occur. Considering your goals for fault tolerance is also important, as they will affect both the architecture of your application and its costs.

▼ The cost of fault tolerance

When architecting fault-tolerant systems, another important consideration is cost. This is a difficult and very case-specific factor, but it's important to remember that while there are costs inherent with choosing and using more fault-tolerant architectures and tools, there are also significant costs associated with *not* choosing a high level of fault tolerance.

For example, operating multiple instances of your database across multiple cloud regions is likely to cost more on the balance sheet than operating a single instance in a single region. However, there are a few things you must also consider:

- **What does an outage cost in dollars?** For mission-critical systems, even a few minutes of downtime can lead to millions in lost revenue.
- **What does an outage cost in reputation damage?** Consumers are demanding, particularly in certain business verticals. An application outage of just a few minutes, for example, could be enough to scare millions of customers away from a bank.
- **What does an outage cost in engineering hours?** Any time your team spends recovering from an outage is time they're *not* spending building new features or doing other important work.

The way you achieve fault tolerance can also have a significant impact on your costs.

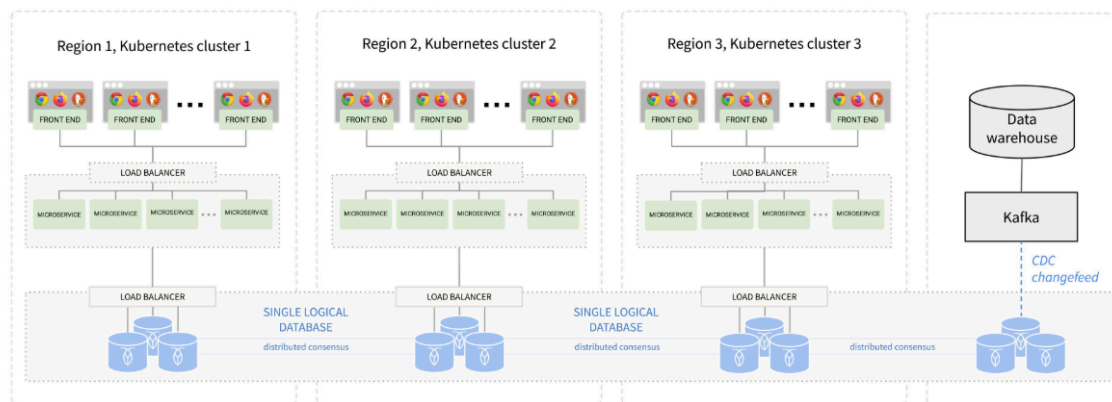
let's consider the real-world case of a major electronics company that needed to build a more scalable, fault-tolerant version of its existing MySQL database.

The company could have made that MySQL database more fault tolerant by manually sharding it, but that approach is technically complex and requires a lot of work to execute and maintain. Instead, the company chose to migrate to CockroachDB dedicated, a managed database-as-a-service that is inherently distributed and fault tolerant.

Although CockroachDB dedicated itself is more expensive than MySQL (which is free), migrating to CockroachDB enabled the company to save millions in labor costs because it automates the labor-intensive manual sharding process and resolves many of the technical complexities that manually sharding would introduce.

This's not to say that CockroachDB or *any* specific tool or platform will be the most affordable option for all use cases. However, it's important to recognize that the methods you choose for achieving your fault tolerance goals can have a significant impact on your costs in both the short and long term.

▼ Fault-tolerant architecture examples



Achieving fault tolerance in the application layer

In the diagram above, the application is spread across multiple regions, with each region having its own Kubernetes cluster.

Within each region, the application is built with microservices that execute specific tasks, and these microservices are typically operated inside Kubernetes pods.

This allows for much greater fault tolerance, since a new pod with a new instance can be started up whenever an existing pod encounters an error.

This approach also makes the application easier to scale horizontally — as the load on a specific service increases, additional instances of that service can be added in real time to handle the load, and then removed when the load dies down again and they're no longer needed.

Achieving fault tolerance in the persistence (database) layer

CockroachDB is chosen because its distributed, node-based nature naturally provides a high level of fault tolerance and the same flexibility when it comes to scaling up and down horizontally.

Being a **distributed SQL database**, it also allows for strong consistency guarantees, which is important for most transactional workloads.

CockroachDB also makes sense for this architecture because although it's a distributed database, it can be treated like a single-instance Postgres database by the application — almost all the complexity of distributing the data to meet your application's availability and survival goals happens under the hood.

▼ Manual sharding and the costs of scaling to address demand

scaling a relational database horizontally requires partitioning (sharding) it into multiple instances so that the growing workload can be distributed across multiple nodes. And since there are finite limits to the performance gains that can be achieved through vertical scaling, scaling out horizontally is almost always required.

However, manually sharding a database is a time-consuming, manual, and complex process that requires additional things for your team to do:

- Design a table sharding (partitioning) model.
- Develop application logic to route data to the correct shard.
- Develop application logic to perform joins of data across multiple table shards.

- Develop application logic to perform atomic transactions across multiple table shards or design the application to avoid needing cross-shard atomic transactions.
- Develop application logic to handle failure scenarios where some shards or partitions are not available.

Manual sharding, in other words, requires making *significant* changes to the code of your application itself — **every part of your application that interacts with the database will need to be reworked**. Other elements of your stack that touch the database, such as data pipelines, will also need to be reworked.

And the costs associated with manual sharding are not a one-time expense — all of this work will need to be repeated the next time you need to scale up. As more shards are added, the system also becomes increasingly complex and brittle, making sharding and managing the database more and more labor intensive as time goes on.

▼ The costs of scaling to address new markets or regions

There are a wide variety of reasons why businesses choose to deploy their databases across multiple geographical regions. Multi-region deployments can decrease latency for users and increase the resilience of the overall systems. They are also sometimes required to ensure compliance with localized privacy and sovereignty regulations (such as GDPR).

Traditional relational databases, however, were designed for single-instance, single-region deployment. Deploying a traditional RDBMS across multiple regions means sharding it *every time you move into a new region*. Since sharding is such a labor-intensive process, this massively increases labor costs.

▼ Understanding the total cost of ownership for cloud databases

Having a database that automates scaling and many of the other labor-intensive operational tasks enables companies to recapture **engineering hours and redirect them to value-generating work** for the business (such as feature development).

conclusion: upgrading to a modern distributed SQL database, despite coming with higher licensing costs, **would actually save the companies millions of dollars.**

▼ 3 reasons to build multi-region application architecture

Multi-region application architecture makes applications more resilient and improves end-user experiences by keeping latencies low for a distributed user base. This blog will dive into each of the top five reasons why multi-region application architecture is worth building

▼ 1. Fault tolerance in multi-region application architecture

Fault tolerance is the ability of a system to endure a failure of some kind and continue to operate properly.

The unfortunate reality is that failures happen. A fire, a flood, or an extreme storm can take out a data center, an availability zone, or an entire region. But that doesn't have to result in an outage.

Multi-region applications are resilient in a way that single region and single availability zone applications are not. Single region architecture can survive machine failures. Single availability zone architecture can survive an AZ failure.

But only a multi-region application architecture can survive a region failure. If your application has high-availability requirements then multi-region architecture is important.

▼ 2. Low latency for geographically dispersed users

How do you get low latency when users are spread out? Multi-Region application architecture helps deliver low latency by making it possible to keep data close to users even when those users are distributed all over the world. Consider an example:

Imagine your business has users in New York City and Sydney, Australia. If you just deploy your application in a data center near New York, then users in Sydney will experience high latencies. They have to wait while their requests travel all the way to New York and back. But if you deploy your applications in multiple regions, and you put another

data center closer to Sydney, then the client request time will dramatically improve.

A good rule of thumb is to keep latencies under 100 milliseconds. That is the maximum threshold for an experience to feel instantaneous.

The only way to achieve low latency, and great user experiences, is to keep the data close to the users with multi-region application architecture.

▼ 3. Comply with data privacy laws & regulations

There are now over 200 data privacy and data storage regulations around the world, the most famous probably being GDPR. And all the signs indicate that there will be more of these regulations every year. These regulations often prohibit storing certain data outside of certain boundaries. With the right multi-region deployment, you can keep data where it legally needs to be, by pinning data within the boundaries of a physical location.

Untitled